



OWASP
Smart Contract Security

scs.owasp.org

Security Handbook : UX Security

09

Security Handbook Series

GRANT SUPPORT • 2025-2026



ecosystem
support
program

Supported by Ethereum Foundation's
Ecosystem Support Program (ESP)



Contents

Frontispiece	4
About This Handbook	4
Copyright and License	5
Author and Creative Credits	5
Purpose and Scope	6
Target Audience	6
How to Use This Handbook	6
Relationship to SCSVS, SCSTG, and SCWE	7
Part I: Foundations	8
1. Introduction to UX Security	8
2. Threat Model for UX	12
Part II: UX Security Scoring Matrix	23
3. Scoring Matrix Overview	23
4. Dimension: Transaction and Action Clarity	27
5. Dimension: Confirmation and Consent	29
6. Dimension: Revocability and Recovery	30
7. Dimension: Consistency and Predictability	32
8. Dimension: Warnings and Error Handling	33
9. Additional Dimensions (Expandable)	35
Part III: Best Practices	40
10. Wallet and Signing UX	40
11. dApp and Connection UX	47
12. Approval and Allowance UX	49
13. Notification and Alert UX	51
14. Onboarding and Education	52
Part IV: Implementation and Evaluation	54
15. Integrating UX Security into Development	54
16. Review and Audit Using the Matrix	59
17. Iteration and User Feedback	63

Part V: Appendices	68
18. Appendix A: Glossary	68
19. Appendix B: UX Security Scoring Matrix (Reference)	72
20. Appendix C: Checklist and Quick Wins	75
21. Appendix D: References and Further Reading	80
Appendix C: References and Further Reading	81
Standards and Frameworks	81
Incidents and Case Studies	81
Tools and Documentation	81
Further Reading	82

Frontispiece

OWASP SMART CONTRACT SECURITY PROJECT

UX Security

About This Handbook

UX Security is part of the OWASP Smart Contract Security Handbook Series. The series extends smart contract security beyond contract code into the people, process, application, infrastructure, and operational layers surrounding Web3 systems.

Each handbook is designed as a defensive field reference for architects, engineers, security practitioners, incident responders, auditors, and organizational leaders. It complements the OWASP Smart Contract Security Verification Standard (SCSVS), Smart Contract Security Testing Guide (SCSTG), Smart Contract Weakness Enumeration (SCWE), Smart Contract Top 10, and SCS Checklist without replacing those resources.

SERIES EDITION **2026**

Copyright and License

Copyright © 2026 The OWASP Foundation and contributors.



This document is released under the [Creative Commons Attribution-ShareAlike 4.0 International License](#). You may share and adapt the material with appropriate attribution and under the same license. For reuse or distribution, make the license terms clear and identify material changes.

OWASP publications are community-developed educational resources. References to organizations, products, or services do not constitute endorsement, and the material is provided without warranty.

Author and Creative Credits

PROJECT LEAD

Shashank | CredShields

CEO and Co-Founder

AUTHOR | PROJECT MAINTAINER

Pratik Lagaskar

Security Researcher

COVER PAGE DESIGNS

Siddharth Neekher

Lead UI Designer @ CredShields

OWASP Smart Contract Security Project

Purpose and Scope

This handbook treats the user interface as a security control, not a coat of paint over a smart contract. It covers user experience (UX) security for Web3 applications and decentralized applications (dApps): reducing user error, preventing phishing through domain spoofing and malicious dApp impersonation, defending against approval and allowance abuse (unlimited allowances, drainer patterns, Permit2 and granular time-limited permissions), and closing the gap that blind signing and multisig user-interface spoofing open, the pattern behind the February 2025 Bybit theft and the front-end compromises that hit Safe-style multisig operators. The scope aligns with the OWASP Smart Contract (SC) Top 10 and the Web3 Attack Vectors taxonomy’s “beyond smart contracts” category, where the vulnerable component is the screen the user trusts, not the bytecode the chain executes.

A contract can pass every audit and still cost its users their funds, because a user never approves calldata: they approve whatever the wallet or dApp shows them on screen, and an attacker who controls that screen controls the outcome regardless of what the contract itself does correctly. This handbook gives design and development teams a concrete UX Security Scoring Matrix and a set of actionable best practices to close that gap, from wallet and signing flows through connection UX, allowance management, notifications, and onboarding.

Target Audience

Product and UX designers, front-end developers, and security practitioners responsible for dApp and wallet interfaces. Wallet teams building signing flows and hardware confirmation screens, and auditors extending contract-level reviews to the interface layer, will find the scoring matrix in Part II and the checklist in the appendices the fastest path to actionable output.

How to Use This Handbook

Start with Part I to establish why UX and security are interdependent and to walk the UX-specific threat model: phishing, allowance abuse, blind signing, and the mismatch between what a transaction’s calldata says and what the interface displays. Part II is the handbook’s core reference, a scoring matrix with dimensions for clarity, confirmation, revocability, consistency, and error handling, each with concrete criteria a team can apply to a real product or feature. Part III turns the matrix into practice with best practices for wallet, connection, allowance, notification, and onboarding UX. Part IV covers integrating these controls into a development lifecycle and re-auditing with the matrix over time. Teams under deadline pressure can jump straight to the scoring criteria in Part II and the quick-wins checklist in Appendix C.

Relationship to SCSVS, SCSTG, and SCWE

The Smart Contract Security Verification Standard (SCSVS) defines the properties a secure system must satisfy at the contract layer; the UX controls in this handbook are the interface-layer counterpart, because a verified contract still fails its users if the signing flow that invokes it is deceptive. The Smart Contract Security Testing Guide (SCSTG) describes how to test contract behavior; Part IV extends that discipline to interface review by turning the scoring matrix into a repeatable audit and iteration process. The Smart Contract Weakness Enumeration (SCWE) catalogs weaknesses in contract code; the phishing, allowance-abuse, and blind-signing failure modes in Part I fill the space the enumeration leaves at the human-computer boundary, where the weakness is in what a user was shown rather than in what the contract executed. The companion Web3 Attack Vectors Mapping Handbook (10) places these interface risks in the broader attack-vector taxonomy, and the CDN and Front-End Supply Chain Security Handbook (01) owns the delivery-layer integrity controls, such as Subresource Integrity and Content Security Policy, that keep the interface this handbook evaluates from being tampered with before it ever reaches the user.

Part I: Foundations

This part establishes why the interface, not only the bytecode, decides whether a Web3 user keeps their funds. It defines the vocabulary this handbook uses for user experience (UX) security, then builds the threat model: the phishing, allowance-abuse, blind-signing, and calldata-mismatch patterns that later parts turn into a scoring matrix and concrete design practice. Every attack technique named here is paired with a detection signal and a mitigation, because a defender needs the mechanism only to the depth required to threat-model, design against, and instrument for it.

1. Introduction to UX Security

1.1 Why UX and Security Are Interdependent

No user ever approves calldata. A user approves whatever the wallet or the decentralized application (dApp) draws on screen: a recipient, an amount, a button labeled “Confirm.” The Ethereum Virtual Machine executes the calldata regardless of whether it matches that label, and once a signature exists, the transaction is final. This is the structural fact this handbook is built on: security in Web3 is not only “does the contract behave correctly,” it is also “does the interface tell the truth about what the contract will do.” A contract can pass every audit line by line and still cost its users everything if the screen between the user and that contract lies.

The gap between what a transaction actually does and what a user believes it does is the attack surface this handbook maps. That gap can open at several points: a wallet that shows a raw hex string instead of a decoded action (blind signing), an interface that renders correct information but was itself served from a compromised host (front-end tampering, the subject of the CDN and Front-End Supply Chain Security Handbook, 01), or a design that shows accurate information in a way a rushed user will not read (a confirmation dialog buried in low-contrast text below a large green button). All three produce the same outcome: a signature the user did not mean to give.

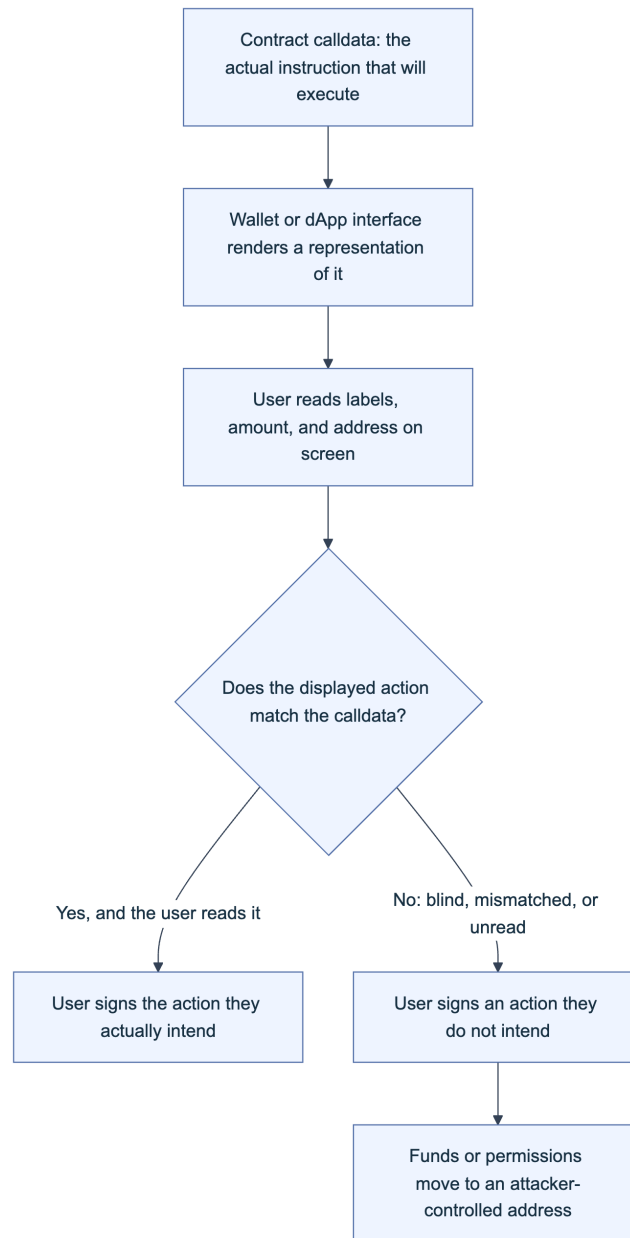


Figure 1. The signing chain. Every control in this handbook exists to keep the path from calldata to user intent unbroken, and to close it fast when it breaks.

Irreversibility raises the stakes on every point where this chain can fail. A mis-clicked confirmation in a Web2 banking app can usually be reversed with a phone call; a signed Ethereum transaction cannot. That asymmetry is why UX defects that would be minor annoyances elsewhere (a confusing settings page, an unclear error message) become fund-loss events in Web3, and why this handbook treats interface design as a security control with the same rigor the rest of the OWASP Smart Contract Security (SCS) Project applies to code.

1.2 Common UX-Related Security Failures in Web3

Failures in this category share a shape: the underlying contract logic can be entirely correct, and the user still loses funds because of what the interface asked them to sign, or failed to warn them about. The table below anchors each recurring failure mode to a concrete, cited incident so the pattern is not abstract.

Failure mode	Mechanism	Representative incident
Blind signing on hardware wallets	Device shows a raw hash or truncated data instead of a decoded action, so the signer cannot verify intent	Bybit/Safe{Wallet} incident, February 2025, FBI/IC3 PSA250226 (Section 2.4)
Unlimited or standing token approvals	<code>approve()</code> calls request unbounded spending rights that outlive the transaction that needed them	Drainer-as-a-service campaigns tracked industry-wide; Scam Sniffer 2025 phishing report (Section 2.2)
DNS and front-end hijack	A cloned interface is served from a hijacked domain and prompts a malicious approval	Curve Finance DNS hijack, August 2022, \$573,000 lost (Section 2.1)
Fake interview and remote-access social engineering	Attacker poses as an employer or partner, delivers malware through a video-call or hiring pretext	Cataloged as WA05 in the OWASP Web3 Attack Vectors Top 15 (Section 2.6)
Calldata-to-display mismatch	Interface renders a benign summary while the signature actually authorizes a different, attacker-chosen action	Root cause of the Bybit/Safe incident and the general class OWASP catalogs as WA06, UI/UX Spoofing and Approval Phishing (Section 2.7)

None of these failures require a bug in the target contract's Solidity. Each one is closed, in whole or in part, by an interface decision: whether to decode calldata before display, whether to default an approval to a bounded amount, whether to verify a domain before rendering a connect prompt, whether to make a warning unmissable rather than merely present. Part 2 turns these decisions into scored, auditable criteria.

1.3 Relationship to SCSVS, SCSTG, SCWE

The OWASP Smart Contract Security Verification Standard ([SCSVS](#)) states the verifiable requirements a contract-layer system must satisfy. This handbook is the interface-layer complement: a system is not secure because the SCSVS checklist is green if the wallet screen that invokes the verified contract shows the user a lie. The Smart Contract Security Testing Guide ([SCSTG](#)) describes procedures for testing contract behavior; Part 4 of this handbook extends that testing discipline outward, turning the UX Security Scoring Matrix (Part 2) into a repeatable interface audit that a team runs the same way it runs a contract test suite. The Smart Contract Weakness Enumeration ([SCWE](#)) catalogs weaknesses in contract code; the phishing, allowance-abuse, blind-

signing, and calldata-mismatch failure modes in this Part fill the space that catalog leaves open, because the weakness in each case is not in what the contract executed but in what the user was shown before they authorized it.

Reference standard	What it governs	Where this handbook attaches
SCSVS	Verifiable contract-layer security requirements	Adds interface-layer requirements (Part 2 matrix) alongside the contract checklist
SCSTG	Contract testing procedures	Part 4 extends the same testing discipline to interface review and re-audit
SCWE	Catalog of contract-code weaknesses	This Part's threat model fills the human-computer boundary the catalog does not reach
OWASP Web3 Attack Vectors Top 15	Non-contract attack taxonomy ("beyond smart contracts")	WA01, WA04, WA05, WA06, WA08, WA13, and WA14 map directly onto Sections 2.1 through 2.7

The [OWASP Smart Contract Top 10: 2026](#) makes the same point at the headline level: its authors note that the largest 2025 losses in Web3 came disproportionately from off-chain and operational failures rather than contract bugs alone, drawing on an analysis of 122 incidents totaling roughly \$905.4 million. UX security is one of the largest components of that off-chain surface, alongside the front-end supply chain (Handbook 01) and operational security (Handbook 11).

1.4 How to Use This Handbook

Read Part 1 once, in order, to internalize the threat model: it is the shared vocabulary every later chapter assumes. After that, treat the handbook as a reference you return to by role and task rather than a document you read cover to cover.

Part	Best read by	When to use it
Part 1: Foundations	Everyone touching a wallet or dApp interface	Once, before any scoring or design work, to fix the threat model
Part 2: UX Security Scoring Matrix	Product owners, security leads, auditors	When evaluating a specific product, feature, or flow against concrete criteria
Part 3: Best Practices	Designers, front-end and wallet engineers	When implementing wallet, connection, allowance, notification, or onboarding UX
Part 4: Implementation and Evaluation	Engineering leads, auditors	When wiring the matrix into a development lifecycle and running repeat audits
Part 5: Appendices	Everyone	As a quick-reference glossary, matrix reference, and quick-wins checklist

A team under deadline pressure can jump straight to the scoring criteria in Part 2 and the quick-wins checklist in Appendix C, but the shortcut only works once the reader has internalized why each criterion exists, which is the job of this Part. Wallet teams building signing flows should pay particular attention to Sections 2.3 and 2.4 and to Chapter 10 in Part 3; teams building a dApp front end that requests approvals should prioritize Sections 2.2 and 2.5 and Chapter 12.

2. Threat Model for UX

2.1 Phishing and Impersonation (Malicious dApps, Domain Spoofing)

Phishing in Web3 inherits every Web2 technique (typosquatted domains, malicious search and social ads, compromised support channels) and adds one that has no Web2 equivalent in severity: a successful phish does not steal a password that can be reset, it extracts a signature that moves value irreversibly. OWASP catalogs this as WA08, general phishing and social engineering, and as WA06, UI/UX spoofing and approval phishing, in the [Web3 Attack Vectors Top 15](#).

Domain-level attacks are the most common entry point. An attacker can register a look-alike domain, buy a search ad that outranks the legitimate result, or, in the more severe case, hijack the legitimate domain's DNS records so that the real URL itself resolves to a cloned front end. The August 2022 [Curve Finance DNS hijack](#) is the reference incident for the last pattern: an attacker altered curve.fi's DNS so the domain pointed to a cloned interface, which prompted visitors to grant a malicious contract spending approval. Eight victims lost roughly \$573,000 in stablecoins before the team redirected users to an alternate domain and urged mass approval revocation.

Vector	How it works	Detection signal	Primary mitigation
Typosquatted or look-alike domain	Registers a domain one character or one transposition off the real one	URL differs on close visual inspection; bookmark or password manager does not autofill	Domain and registrar lock, DNSSEC (Handbook 02); bookmark canonical URLs, never follow search results
Malicious search or social ads	Buys ad placement above the organic legitimate result	Sponsored-result label; URL in the ad does not match the brand's known domain	User education; brand monitoring and ad-network takedown requests
DNS hijack of the real domain	Compromises registrar or DNS provider so the legitimate URL serves attacker content	Unexpected wallet-connect prompt or approval request on a previously trusted page	Registrar lock, DNSSEC, monitoring for DNS record changes (Handbook 02)
Malicious dApp mimicking a known brand	Clones a legitimate dApp's interface pixel-for-pixel on unrelated infrastructure	Wallet flags the origin as unverified or unknown	Domain verification at connection time, such as the WalletConnect Verify API



Figure. A real Web3-specific deception pattern: verified-looking identity, impossible benefit, borrowed Ethereum branding, and a call to action that moves the victim from a social feed to attacker-controlled transaction UX. Reviewers should test the entire trust path, not merely the final wallet dialog. Source: ethereum.org, *Security and scam prevention*, CC BY 4.0.

The **WalletConnect Verify API** is the concrete control worth naming here because it operationalizes domain trust at the connection step rather than leaving it to the user's memory. It checks the requesting domain against a registry and security-partner scanners, then returns one of four states to the wallet: **VALID** (the domain matches a registered, unflagged application), **INVALID** (the domain does not match registered records but is not known malicious), **UNKNOWN** (unverified, no detected threat), or a **Threat** state that sets `isScam` true. The wallet surfaces this as a warning the user must acknowledge before connecting. The project's own documentation is honest about its limits: it is "not designed to be bulletproof but to make the impersonation attack harder," which is the correct way to reason about any single anti-phishing control in this handbook. It raises cost, it does not remove risk, so it belongs alongside domain hardening (Handbook 02) and user-facing warnings (Section 2.5), not in place of them.

2.2 Approval and Allowance Abuse (Unlimited Allowances, Drainer Patterns)

The ERC-20 token standard's `approve` and `allowance` functions ([EIP-20](#)) let a token owner authorize a spender, typically a contract, to move up to a specified amount on their behalf. This is legitimate and necessary: a decentralized exchange cannot swap a user's tokens without permis-

sion to move them. The UX failure is not the mechanism, it is the default. Interfaces routinely request the maximum possible allowance, $2^{256} - 1$, framed to the user as convenience: approve once, never see the prompt again.

```
// The pattern that appears in the vast majority of dApp "Approve" buttons.
// The user is told this saves gas on future transactions.
// The user is not told the approval never expires and is not capped to the trade
amount.
IERC20(token).approve(spender, type(uint256).max);
```

The risk that default creates is durational, not transactional. A one-time swap leaves a standing, unlimited allowance on that token indefinitely, for every spender the user has ever approved, until manually revoked. A drainer, malicious code distributed as a phishing-site payload or a compromised dApp update, does not need to steal a private key: it only needs the user to sign one more approval or `Permit` message, then it calls `transferFrom` against every allowance it can find. This is the mechanism the [Scam Sniffer 2025 phishing report](#) tracks at scale: the firm reports 2025 phishing losses of roughly \$84 million, an 83 percent year-over-year decline it attributes partly to wider adoption of the defenses this section describes, against a detection pipeline that scanned 47 million URLs and identified 705,000 scams. OWASP catalogs the packaged toolkits behind this activity as WA04, Drainer Malware and Drainer-as-a-Service, in the [Web3 Attack Vectors Top 15](#).

A user (or an auditor reviewing a wallet's exposure) can check a standing allowance directly against the chain rather than trusting any dashboard:

```
# Query the current allowance a wallet has granted to a spender contract
cast call $TOKEN_ADDRESS "allowance(address,address)(uint256)" $OWNER_ADDRESS $SPENDER_ADDRESS --rpc-url $RPC_URL

# Revoke it by setting the allowance back to zero
cast send $TOKEN_ADDRESS "approve(address,uint256)" $SPENDER_ADDRESS 0 --rpc-url $RPC_URL --private-key $PK
```

[EIP-20](#) itself flags a related race condition and recommends that “clients SHOULD make sure to create user interfaces in such a way that they set the allowance first to 0 before setting it to another value for the same spender,” because a spender who observes a pending increase-allowance transaction can front-run it and spend both the old and new amounts. The design implication for interfaces is threefold: default to the exact amount a transaction needs rather than an unbounded maximum, surface existing standing allowances prominently rather than burying them in a settings page, and make revocation a one-click action rather than a multi-step chain-explorer exercise. Part 3, Chapter 12 turns these into scored criteria.

2.3 Permit2 and Signature-Based Approvals vs. Legacy Allowances

Permit2, a contract Uniswap Labs published and other protocols have since adopted as a shared primitive, was designed specifically to close the durational risk Section 2.2 describes. It unifies two mechanisms in a single contract: `SignatureTransfer`, which authorizes a one-time transfer through an off-chain signature whose permission “only lasts for the duration of the transaction that the one-time signature is spent,” and `AllowanceTransfer`, which grants a spender permission for a specified amount over an explicitly bounded duration rather than indefinitely.

The security-relevant difference from legacy `approve` is not cosmetic. A legacy approval is an on-chain transaction that persists as contract state until a second on-chain transaction revokes it; nothing about the token contract enforces an expiry. A Permit2 `SignatureTransfer` authorization is a signed message, not a state change, and it is spent, not standing: once used or once its deadline passes, there is nothing left for a drainer to find. A Permit2 `AllowanceTransfer` grant does persist, but with a duration baked into the grant itself, so an abandoned approval ages out instead of remaining exploitable indefinitely.

Dimension	Legacy <code>approve</code> (EIP-20)	Permit2 <code>SignatureTransfer</code>	Permit2 <code>AllowanceTransfer</code>
Authorization mechanism	On-chain transaction	Off-chain signature, consumed on use	Off-chain signature grants a bounded on-chain allowance
Default duration	Indefinite until revoked	Single transaction	Explicit, bounded expiration
Standing risk after use	Persists at whatever amount was approved	None; nothing remains to reuse	Persists only until the stated expiry
Revocation	Separate on-chain transaction required	Not applicable, nothing to revoke	Separate on-chain transaction, or natural expiry
Phishing surface	A single signed approval can be reused indefinitely by anything with spender rights	A stolen signature is worthless after first use or deadline	A stolen signature grants only a time-boxed, capped allowance

This does not make signature-based approval risk-free, and the UX implication for this handbook cuts both ways. A `Permit` or Permit2 signature request looks, on most wallet screens, exactly like any other message to sign: it does not trigger the same visual weight as an on-chain transaction, so an interface that fails to decode and clearly label what is being authorized can make a Permit2 phish look like an innocuous “sign in” prompt. The mitigation is the same one this handbook returns to throughout Part 2: the signing interface must render the spender, the amount, and the expiry from the structured [EIP-712](#) message in plain language, not just confirm that a signature was requested. Section 2.4 covers what happens when that rendering step is itself compromised.

2.4 Blind Signing and Multisig UI Spoofing (Bybit/Safe-Style Risks)

Blind signing means approving a transaction the signing device cannot render in human-readable form, typically because the device shows only a raw hash or an unparsed data blob rather than a decoded function call. [Ledger's own guidance](#) describes the risk directly: users approving data “you can't fully read or instantly understand” are, in effect, signing a blank check, and it states that “billions of dollars have been siphoned from unsuspecting users this way.” The countermeasure the industry has converged on is Clear Signing, formalized for the Ethereum Virtual Machine ecosystem as [ERC-7730](#), a JSON-based standard that lets a wallet or dApp supply the metadata needed to translate calldata and [EIP-712](#) typed data into a display like “Send 100 USDC to alice.eth” instead of a hex string, with the file also encoding constraints that bind the display to the correct contract and chain.

The February 2025 theft from Bybit is the reference incident for what happens when this gap is exploited deliberately rather than merely tolerated by default. The [FBI's Internet Crime Complaint Center](#) attributed the theft, roughly \$1.5 billion in virtual assets, to North Korea, identifying the operation as TraderTraitor and noting the funds were rapidly converted and laundered across thousands of addresses. Independent investigation by the incident-response firm Sygnia and reporting that followed it reconstructed the mechanism: attackers compromised a [Safe{Wallet}](#) developer's workstation through a social-engineered malicious project, then used harvested AWS credentials to move inside Safe{Wallet}'s cloud infrastructure for roughly two weeks before injecting malicious JavaScript into the production web application.

Date	Event
Feb 4, 2025	Safe{Wallet} developer's macOS workstation compromised via a social-engineered malicious Docker project
Feb 5-17, 2025	Attacker operates inside Safe{Wallet}'s AWS infrastructure, timing activity to the developer's working hours and routing through VPN infrastructure to evade detection
Feb 19, 2025	Attacker modifies JavaScript in Safe{Wallet}'s production S3-hosted bundle, targeting Bybit's cold-wallet transaction specifically
Feb 21, 2025	Bybit signers execute a routine cold-to-warm transfer; malicious code replaces the transaction payload with a delegatecall to an attacker contract exposing sweepETH and sweepERC20
Within ~2 minutes of execution	Attacker removes the malicious JavaScript from the production interface, restoring its normal appearance

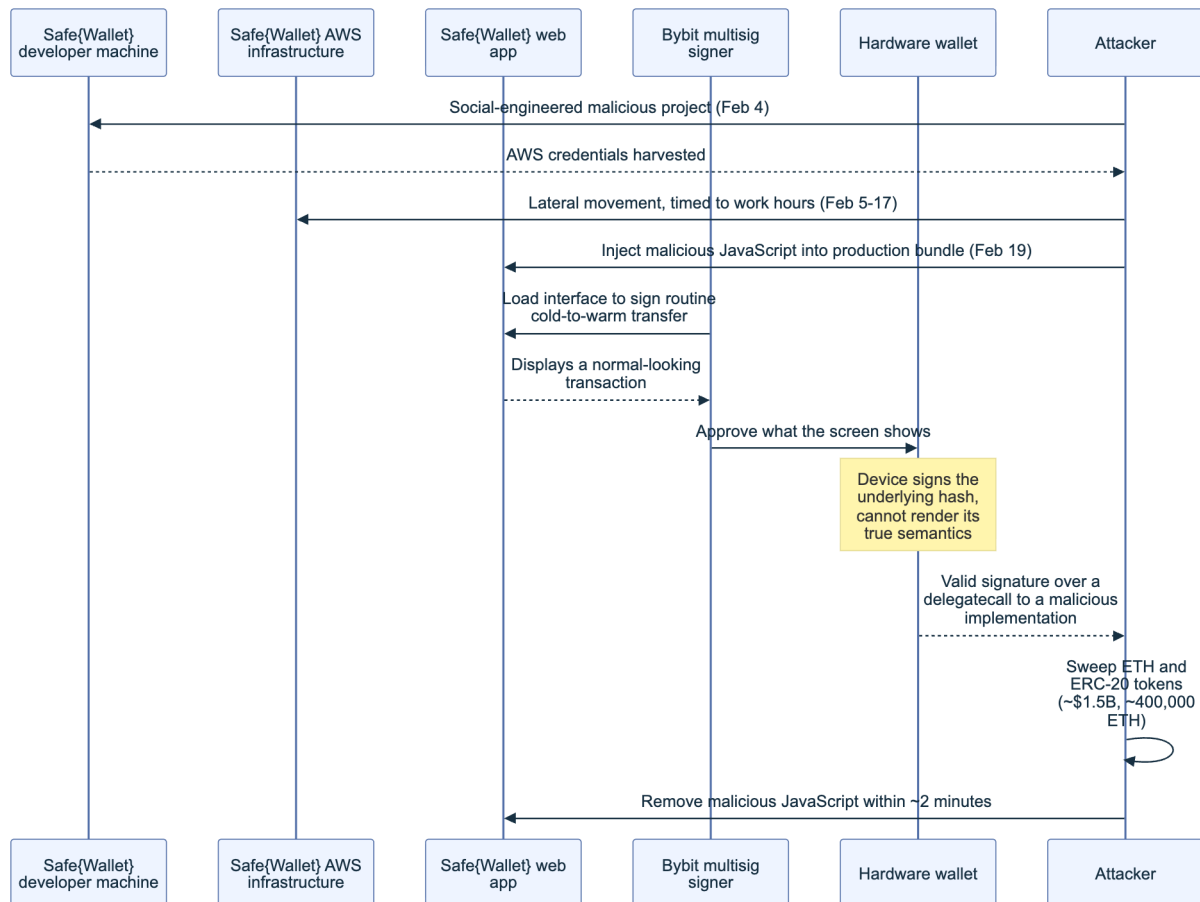


Figure 2. The Bybit/Safe{Wallet} attack sequence. The compromise never touched a smart contract; it touched the interface layer that decides what a signer believes they are approving. Sources: [FBI IC3 PSA250226](#), [Sygnia investigation](#).

The specific technical trick, replacing the transaction payload with a delegatecall that swapped in a malicious implementation exposing sweep functions, worked because the signers were relying on the web interface to tell them what they were approving, and hardware wallets in the loop displayed the resulting hash rather than a decoded, semantically meaningful summary. This is precisely the blind-signing gap ERC-7730 targets and precisely the multisig-hijacking pattern OWASP catalogs as WA01 in the [Web3 Attack Vectors Top 15](#), describing attackers who manipulate signing interfaces through “blind signing, UI spoofing, and malicious JavaScript injected into wallet frontends.” Bybit remained solvent by securing bridge financing from Galaxy Digital, FalconX, and Wintermute and replacing roughly 447,000 ETH within 72 hours, but that recovery was a balance-sheet response, not a control that would have prevented the loss. The controls that would have are the ones this handbook develops: clear signing on the hardware device, independent verification of calldata against display (Section 2.7), and multisig operational practices covered in Handbook 11.

2.5 Confusion and Misleading Flows

Not every UX security failure requires an attacker at all. A confusing flow can produce the same fund-loss outcome as a phish, purely through design that a rushed or unfamiliar user misreads, and a flow can also be confusing by deliberate design, following the same manipulative patterns known in consumer interface design as deceptive or dark patterns. The **Deceptive Design taxonomy** (curated by Harry Brignull, whose 2010 coinage of “dark patterns” the field still uses) names the recurring forms; several map directly onto wallet and dApp flows.

Deceptive pattern	Wallet/dApp instance	Security consequence
Sneaking	A “max approval” or “unlimited allowance” checkbox is pre-selected, or bundled invisibly into a “Continue” flow	User grants standing risk (Section 2.2) without deliberate choice
Fake urgency	“This price expires in 10 seconds, sign now” pressures a rushed signature	User skips reading the transaction summary or hardware-device display
Trick wording	A button labeled “Approve” or “Continue” is the only visible label for what is actually an unlimited-allowance grant	User cannot distinguish a bounded action from an unbounded one
Visual interference	Approval amount, spender address, or expiry rendered in small, low-contrast text below a prominent confirm button	Critical details are present but practically unreadable
Forced action	A page gates unrelated content (an airdrop claim, a “mint” reveal) behind a wallet connection or signature	User signs to unblock content, not because they evaluated the request

Hurry up! Sale ends in
00d 00h 12m 34s

Only 5 coupons left

**SIGN UP FOR
FREE COUPON**

67 friends have signed up

I'm boring and like paying full price

☒ I don't want no marketing email

Figure. Deceptive Design's canonical pop-up example, annotated with the same manipulative techniques that show up in wallet and dApp flows: pressure to act fast, an opt-out buried in low-contrast text, and wording that hides what a click actually commits to. Source: [Wikimedia Commons](#), CC BY-SA 4.0.

The defense against confusion is structurally different from the defense against phishing, because there is no external domain or spoofed brand to detect; the interface asking for the signature is the legitimate one. The remedy is design discipline: put the security-relevant facts (amount, recipient, expiry, revocability) in the most visually prominent position in the flow, never the least, use consistent terminology for the same action across every screen so a user's learned pattern-matching stays reliable, and never gate content behind a signature that is not required to render that content. Part 2, Chapter 7 (Consistency and Predictability) and Chapter 4 (Transaction and Action Clarity) turn these principles into the scored criteria a team can apply to a specific flow. NIST's usable-security research reaches the same conclusion from the defender's side of the equation: a security decision a user cannot parse in the time they are willing to spend on it is not a decision, it is a coin flip weighted toward whatever the interface makes easiest.

2.6 Information Disclosure and Social Engineering

UX security failures are not limited to signature theft. An interface can leak information that itself enables an attack, or it can be the delivery surface for social engineering that never touches a signing prompt directly. OWASP's [Web3 Attack Vectors Top 15](#) catalogs both patterns: WA08 covers general phishing and social engineering targeting seed phrases and credentials, increasingly

assisted by AI-generated content, and WA05 covers a specific and now well-documented pattern of fake interview and video-call social engineering, in which an attacker poses as an employer or business contact, moves the conversation to a video call, and abuses remote-desktop or file-sharing features to install malware that steals wallet material. This pretext is closely associated with North Korea-linked activity against cryptocurrency-industry employees and job seekers, consistent with the FBI's broader public warnings about state-linked social engineering against the sector.

Interface-level information disclosure compounds this risk even without malware. Common leaks include: error messages or “insufficient balance” states that confirm a wallet address holds funds before an attacker commits to targeting it; transaction-history views that expose a user's full counterparty graph to anyone who loads the page, not only the account owner; browser autofill or clipboard-history integration that can surface a seed phrase fragment typed into the wrong field; and support-chat widgets that route to third-party services without disclosing that routing, creating a channel an attacker can spoof.

A practical checklist for reviewing a flow against this section:

- Does any user-facing error or state message reveal wallet contents or activity to an unauthenticated viewer?
- Does the interface ever ask for a seed phrase, private key, or full mnemonic in any field, including “support” or “verification” flows? (The correct answer is always no; a legitimate wallet or dApp never asks.)
- Are support channels (chat, Discord, Telegram) clearly labeled as official, and does the interface warn users that support staff will never DM first or ask for keys?
- Does any onboarding or KYC step request more personal information than the stated purpose requires, creating a target for downstream doxxing or SIM-swap-enabled attacks?
- Are remote-access or screen-sharing requests ever presented as a normal part of a legitimate support or partnership flow? (They should not be; this is the WA05 pretext.)

2.7 Transaction Calldata vs. UI Mismatch (Verify Independently)

Every failure mode in this chapter converges on one final question: does the transaction the user is about to sign actually do what the screen says it does? Sections 2.3 and 2.4 showed that the interface itself, whether through a phished Permit2 message or a compromised Safe{Wallet} front end, can be the source of a mismatch between display and reality. The defense is to treat the on-screen summary as a claim to be verified, not a fact to be trusted, whenever the value at stake justifies the extra step, which for a multisig treasury transaction or a large approval should be always.

Independent verification means checking the raw calldata against an independent source before signing, not re-reading the same interface that might be compromised. A user or reviewer can decode calldata directly rather than trusting the dApp's rendering:

```
# Decode a function selector to confirm what a transaction is actually calling
cast 4byte 0xa9059cbb
# => transfer(address,uint256)

# Decode the full calldata of a pending transaction against the known ABI
cast calldata-decode "transfer(address,uint256)" 0xa9059cbb000000000000000000000000...

# Cross-check the recipient contract's verified source on a block explorer
# to confirm it matches the implementation you expect, not a swapped-in one
```

This checklist operationalizes calldata-versus-display verification for a high-value signature:

- Decode the calldata independently (via `cast`, a block explorer’s “Decode Input Data” view, or an [ERC-7730](#)-aware hardware wallet display) rather than trusting only the requesting interface’s summary.
- Compare the recipient or spender address character-by-character against a known-good source, not just the first and last few characters; address-poisoning attacks specifically target the truncated-address habit.
- For a contract call, confirm the target contract’s verified source matches the implementation you expect, especially for any proxy or Safe-style account where the “same address” can point to a swapped implementation, exactly as in the Bybit case.
- For a signature request (`Permit`, `Permit2`, or any [EIP-712](#) typed message), read the decoded spender, amount, and expiry fields specifically; a signature request that looks like “just logging in” is the highest-risk disguise because it carries none of the visual weight of an on-chain transaction.
- Treat a hardware wallet screen showing only a hash, rather than decoded fields, as a stop condition, not a formality to click through; that is blind signing, and Section 2.4 is what it costs when exploited deliberately.
- Where available, use wallet or browser-extension transaction-simulation and preview features as a second, independent rendering of intent, since a simulation that disagrees with the dApp’s own summary is a strong signal something is wrong.

None of these steps require trusting the interface that generated the transaction in the first place, which is the entire point: the mismatch this section defends against is, by definition, a case where that interface cannot be trusted to accurately describe its own request.

Key controls for Part 1

- Treat the interface as a security boundary equal in weight to the contract it invokes; a correct contract behind a lying screen still costs users their funds.
- Verify domains and dApp origins at connection time (for example, the WalletConnect Verify API) rather than relying on user memory alone.

- Default approvals to the exact amount needed and a bounded duration; prefer Permit2-style time-boxed or single-use authorization over indefinite legacy allowances.
- Treat any hardware-wallet screen showing an undecoded hash as a stop condition; require clear signing (ERC-7730 or equivalent) for any signature moving meaningful value.
- Put security-relevant facts (recipient, amount, expiry, revocability) in the most visually prominent position in a flow, and never gate unrelated content behind an unnecessary signature.
- Independently decode and verify calldata for high-value or multisig transactions rather than trusting a single interface's summary, and treat a decode-versus-display mismatch as a stop condition.

Part II: UX Security Scoring Matrix

Part I named the failure modes: phishing, allowance abuse, blind signing, calldata that does not match what the screen shows. This Part turns those failure modes into an instrument. The UX Security Scoring Matrix gives a product, design, or security team a repeatable way to walk a wallet or decentralized application (dApp, an application whose backend logic runs on a blockchain rather than a controlled server) screen, assign an evidence-backed score to each dimension of risk, and compare that score across releases instead of arguing from intuition about whether a flow “feels safe.”

3. Scoring Matrix Overview

This chapter defines what the matrix is for, what it measures, how scores are assigned, and how a team runs it against a real product surface. Chapters 4 through 9 then work through each dimension in depth with its own rubric.

3.1 Purpose and Use of the Matrix

A statement like “the approval flow feels risky” does not survive a sprint-planning meeting. A statement like “Confirmation and Consent scored 1 of 3 because the unlimited-approval toggle defaults to on” does. The matrix exists to convert the threat model from Part I into scores a team can track release over release, compare across competing products, and use to prioritize engineering time against the dimensions that most directly cause fund loss.

The matrix is deliberately narrower than a general usability audit. Jakob Nielsen’s ten usability heuristics ([Nielsen Norman Group](#)) ask whether an interface is learnable, efficient, and forgiving of ordinary mistakes. A screen can score well on every one of those heuristics and still walk a user into signing an unlimited token allowance for an attacker-controlled contract, because heuristic

usability and security-relevant clarity are different properties. The matrix in this Part scores the second property specifically: whether the interface accurately represents risk, forces deliberate consent for irreversible or high-value actions, and gives the user a path back out.

Three groups use it in practice. Product and design teams run it against a mock or a staging build before a feature ships, treating a low score the same way they would treat a failed accessibility check: a release blocker, not a backlog item. Security reviewers and auditors run it against a shipped product as an extension of a contract-level audit, since a UX gap can turn a technically sound contract into a practical drain vector. Comparative researchers run it across competing wallets or dApps to benchmark the market and identify which pattern deserves to become the default.

3.2 Dimensions (Clarity, Confirmation, Revocability, Consistency, etc.)

The matrix has five core dimensions, each covered in its own chapter, plus two expandable dimensions that a team scores when the product surface makes them relevant. Every dimension asks a distinct security question; none substitutes for another, and a strong score in one does not compensate for a weak score in another. The Consistency dimension in particular does not multiply into the aggregate at full weight, because a predictable interface only reduces attack surface indirectly, by making it harder for a spoofed screen to blend in (Section 9.3 gives the weighting).

Dimension	Chapter	Core security question	Representative failure
Transaction and Action Clarity	4	Can the user see, in plain language, exactly what they are about to sign?	Raw calldata hex with no recipient, amount, or asset decoded
Confirmation and Consent	5	Does the interface force a deliberate, informed step before an irreversible or high-value action?	Unlimited approval bundled silently into a one-click "connect" flow
Revocability and Recovery	6	Can the user see and undo what they granted, and recover safely if something goes wrong?	No in-app approvals dashboard; official-looking support channel accepts direct messages
Consistency and Predictability	7	Does the same action always look the same and lead to the same outcome?	"Approve" on the web app, "Allow" on iOS, "Enable" in a modal, for the identical grant
Warnings and Error Handling	8	Does displayed risk scale with actual danger, and do errors explain what to do next?	Generic "Transaction failed" with no reason and no remediation step

Dimension	Chapter	Core security question	Representative failure
Accessibility and Inclusivity (expandable)	9.1	Can users with disabilities perceive and act on the same security-critical information as everyone else?	A truncated address read by a screen reader as one unannounced token
Onboarding and Education (expandable)	9.2	Does the product teach security concepts at the moment they become relevant?	Seed-phrase warning shown once at signup, never surfaced again

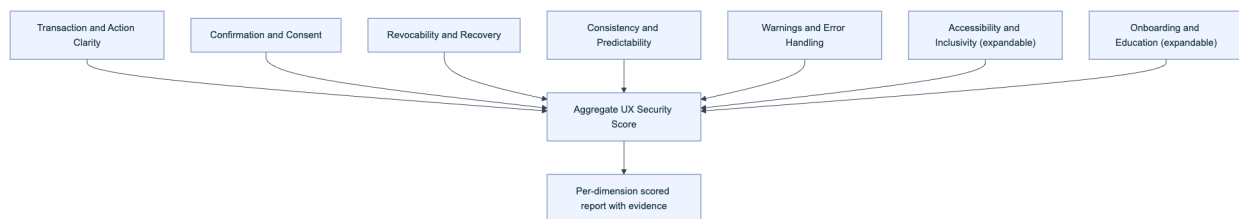


Figure 1. The seven dimensions of the matrix. Five core dimensions and two expandable dimensions each contribute an independent score; none is a proxy for another.

3.3 Scoring Levels and Criteria

Every dimension is scored on the same four-point scale, 0 through 3, so scores are comparable across dimensions and across products. The scale borrows its structure from Jakob Nielsen’s severity-rating method for usability problems (Nielsen Norman Group, “How to Rate the Severity of Usability Problems”), which combines frequency, impact, and persistence into a single 0-4 judgment. This matrix recasts those three factors for a security context: instead of frequency, ask how easily an attacker can steer a user into the gap (exploitability); instead of impact, ask how much value or scope is exposed if the gap is exploited (blast radius); instead of persistence, ask whether the user can undo the damage afterward (recoverability). A dimension scores low when it is easy to exploit, exposes a large blast radius, and leaves the user with no way back.

Level	Name	General criteria	Typical evidence
0	Absent or actively harmful	The control is missing, or the design actively steers the user toward the riskiest option	No confirmation screen before signing; unlimited approval pre-selected as the default
1	Minimal or ad hoc	Some protection exists, but it is inconsistent, easy to miss, or trivially bypassed	A warning is present but styled identically to routine informational text
2	Adequate baseline	Meets what a security-conscious user would expect for the common path through the flow	Plain-language summary shown for a standard swap; revoke option exists but is several taps deep

Level	Name	General criteria	Typical evidence
3	Exemplary, defense-in-depth	Independent layers catch the failure of any single one; matches or exceeds current best practice	Simulation-derived plain language, risk-scaled friction, and a one-tap revoke path all present together

A reviewer assigns a score by citing the specific evidence in the row above, not by impression. Two reviewers scoring the same screen from the same evidence should converge on the same number; where they do not, the criteria description in the chapter's [.3](#) section (for example, 4.3 for Clarity) is the tie-breaker.

3.4 How to Apply to a Product or Feature

Run the matrix as a loop, not a one-time report. Six steps make it repeatable across a team and across releases.

1. **Define scope.** Pick one flow or one screen, such as “the token approval step of the swap flow,” rather than an entire application at once. Scoring the whole app in one pass produces vague scores that nobody can act on.
2. **Walk each dimension against its criteria.** Use the rubric in the matching [.3](#) section (4.3, 5.3, 6.3, 7.3, 8.3, and 9.1 to 9.3 for the expandable dimensions).
3. **Assign a 0-3 score with evidence.** Record the specific screen state, copy, or code path that justifies the number, not a paraphrase of the level description.
4. **Compute the weighted aggregate** using the weights in Section 9.3.
5. **Rank gaps by risk times remediation effort**, prioritizing dimensions where a 0 or 1 score exposes a large blast radius and the fix is cheap.
6. **Ship fixes and re-score the same scope.** The re-score, not the original score, is what goes in the release record.

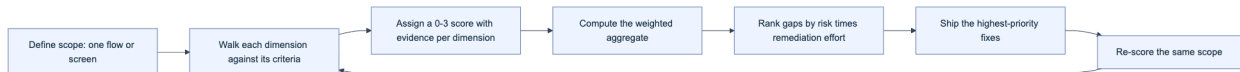


Figure 2. The apply-and-re-score loop. Scoring a static snapshot once has limited value; the matrix earns its keep as a recurring check tied to the release process.

A worked example against a hypothetical token-swap confirmation screen shows the output format:

Dimension	Score (0-3)	Evidence
Transaction and Action Clarity	2	Token amounts and a fiat-equivalent value are shown; no flag for an unverified counterparty contract

Dimension	Score (0-3)	Evidence
Confirmation and Consent	1	The unlimited-approval option is the pre-selected default toggle state
Revocability and Recovery	1	No in-app link to an approvals dashboard; user must find a third-party tool unaided
Consistency and Predictability	2	Copy matches between the web app and browser extension; the mobile app uses a different button label for the same action
Warnings and Error Handling	1	A failed simulation returns a generic revert message with no plain-language cause
Weighted aggregate (core dimensions)	7 of 15 (47%)	Priority fixes: change the approval default to an exact amount, add a one-tap revoke link

4. Dimension: Transaction and Action Clarity

Clarity asks whether the interface tells the truth about what a signature does, in language a non-technical user can act on before, not after, the signature is final.

4.1 Display of Recipient, Amount, and Asset

Every confirmation screen for a value-moving action should answer three questions unambiguously: where is this going, how much, and of what. Each question has a specific way to fail. Recipient display fails when an interface shows only a truncated address (`0x71C7...976F`), because attackers exploit this by generating vanity addresses that share the truncated prefix and suffix and sending a near-zero “dust” transaction from one, hoping the user copies the poisoned address from their own transaction history on a later transfer. The mitigation is to show the full checksummed address alongside any truncated display, and to warn explicitly when a recipient address was typed or pasted rather than selected from a verified contact or Ethereum Name Service (ENS) record. Amount display fails when a raw integer is shown without correct decimal handling. A token with 6 decimals (USDC) displayed as if it had 18 turns a \$500 transfer into an apparent \$0.000000000000005 transfer, or the reverse. Asset display fails through symbol collision: any account can deploy a token contract named “USDC” with a spoofed logo, and an interface that trusts contract-supplied metadata over a curated token list will render the scam token identically to the real one.

Required field	Failure mode	Minimum control
Recipient	Truncated address only; poisoned lookalike accepted silently	Full checksum address available on tap; warn on pasted addresses

Required field	Failure mode	Minimum control
Amount	Wrong decimal precision; scientific notation	Decimal-corrected display plus fiat-equivalent value
Asset	Symbol or logo spoofing from an unverified contract	Cross-check against a maintained token list; flag unlisted contracts

4.2 Plain-Language Summaries

Most transactions and signature requests arrive at the wallet as encoded calldata or a typed-data payload, both of which are unreadable to the overwhelming majority of users. EIP-712, the Ethereum Improvement Proposal (EIP) that standardizes typed structured data for signing, exists specifically to close this gap: it replaces “an opaque hex string displayed to the user with little context” with structured fields a wallet can render in human terms ([EIP-712](#)). A wallet or dApp scores well on this criterion when it decodes the request, whether EIP-712 typed data or raw calldata, into a sentence describing the actual effect, ideally backed by a transaction simulation that executes the call against current chain state before the user signs.

Raw signal	Plain-language rendering
Calldata calling <code>transfer(address,uint256)</code> with a router address	“Transfer 500 USDC to the Uniswap Router contract”
<code>eth_signTypedData_v4</code> request with an unbounded <code>value</code> field and no <code>expiration</code>	“Approve unlimited USDC spending by 0xAbc1...29Fd, with no expiration date”
Call to a proxy contract deployed two hours ago	“Interact with an unverified contract deployed 2 hours ago”

4.3 Scoring Criteria and Examples

Level	Criteria	Example
0	Raw hex or calldata only, no decoding of recipient, amount, or asset	Legacy signing dialogs that show only an opaque message hash
1	Partial decoding; some fields shown, others left raw or mislabeled	Amount shown correctly but recipient still truncated with no full-address option
2	Full plain-language summary for standard, known transaction types	Swap and transfer flows fully decoded; unfamiliar contract calls fall back to raw display
3	Full plain-language summary backed by pre-signing simulation for all transaction types, including unrecognized contracts	Simulation flags an unexpected state change, such as an approval hidden inside a “claim” call, before the user signs

5. Dimension: Confirmation and Consent

Confirmation and Consent asks whether the interface makes the user deliberately choose the risky path, rather than defaulting them onto it. The February 2025 Bybit incident is the reference case for this dimension across the industry: the OWASP Web3 Attack Vectors Top 15 classifies it under multisig hijacking, noting that in the roughly \$1.5 billion theft, attackers compromised the front-end infrastructure the signers relied on so that “signers saw one transaction but executed another” (OWASP Web3 Attack Vectors Top 15, WA01; Bybit background, Wikipedia). No amount of multisig threshold configuration protects a signer who is shown a truthful-looking screen for a different transaction than the one their key actually signs.

5.1 Explicit Confirmation for High-Risk Actions

Not every action deserves the same friction. A read-only balance check should be instant; changing a multisig’s signer set should not be. Define a fixed list of high-risk action categories up front, and require deliberate, hard-to-misclick confirmation for each: unlimited or open-ended token approvals, ownership or admin-role transfers, contract upgrade authorizations, adding or removing a multisig signer or changing its threshold, and any transfer above a configurable value threshold. For these categories, add friction that a phishing clone or a compromised front end cannot silently replicate: require the user to type the exact amount or the last six characters of the destination address rather than clicking a single “Confirm” button, and where the action is irreversible and high value, add a short mandatory delay or an out-of-band confirmation on a second device.

High-risk action	Recommended confirmation mechanism
Unlimited token approval	Type the numeric amount to be approved; reject a default of “unlimited”
Multisig signer or threshold change	Out-of-band confirmation (a call or a second app) in addition to the in-app signature
Large-value transfer	Configurable value threshold triggers a type-to-confirm step and a short delay
Contract upgrade or ownership transfer	Explicit summary of the change with a mandatory waiting period before execution

5.2 Approval and Allowance Flows

The legacy ERC-20 pattern, the Ethereum Request for Comment (ERC) fungible-token standard, grants approvals that are open-ended by default and remain live until manually revoked, regardless of whether the user ever transacts with the spender again. Permit2, a Uniswap-maintained contract, replaces this with signature-based transfers that never create a persistent on-chain approval and with time-bound allowances that expire automatically, combining “signature-based

transfers and time-bound allowances for token spending” ([Permit2 overview](#), [Uniswap developer documentation](#)). Scoring this dimension well means the product defaults to the narrower grant, not the broader one, and surfaces the expiration or scope to the user before they sign.

```
// Legacy ERC-20 pattern: unlimited, indefinite approval is the common default
IERC20(token).approve(spender, type(uint256).max);

// Permit2 pattern: signature-scoped, time-bound allowance
// The user signs an off-chain EIP-712 message; no on-chain approval transaction re-
// quired
IAllowanceTransfer.PermitSingle memory permit = IAllowanceTransfer.PermitSingle({
    details: IAllowanceTransfer.PermitDetails({
        token: token,
        amount: 500e6, // exact amount needed, not
        unlimited
        expiration: uint48(block.timestamp + 30 minutes), // expires automatically
        nonce: nonce
    }),
    spender: spender,
    sigDeadline: block.timestamp + 30 minutes
});
```

5.3 Scoring Criteria and Examples

Level	Criteria	Example
0	High-risk actions confirmed with the same one-click flow as routine actions	Unlimited approval and a routine balance refresh both need only a single tap
1	Some high-risk actions get extra confirmation; coverage is inconsistent	Large transfers require a second tap, but signer-set changes do not
2	All defined high-risk categories require deliberate, distinguishable confirmation	Type-to-confirm required for unlimited approvals and ownership changes
3	High-risk confirmation is out-of-band or otherwise resistant to a compromised front end	Multisig changes require confirmation through an independent channel, not only the same web session

6. Dimension: Revocability and Recovery

Revocability and Recovery asks what happens after something goes wrong, whether that is a stale allowance the user forgot about or a support interaction with someone impersonating the product’s own team.

6.1 Revoking Approvals and Permissions

An allowance a user granted six months ago for a since-abandoned protocol is a live liability until it is revoked, and most users have no visibility into which allowances they still hold. A product scores well here when it exposes its own approvals inside the app, not only through a third-party tool, and when revocation takes one action rather than a multi-screen hunt. External tools remain the fallback for allowances granted outside the product: Etherscan's Token Approval Checker and the independent service Revoke.cash both let a user “inspect and revoke token approvals” across a connected wallet and, in Revoke.cash's case, across more than 100 networks ([Revoke.cash](https://revoke.cash); Etherscan Token Approval Checker, etherscan.io/tokenapprovalchecker). Products should link to these, or to an in-house equivalent, directly from the confirmation flow that created the allowance in the first place.

```
// Manual revoke of a legacy unlimited approval
IERC20(token).approve(spender, 0);

// Permit2 allowances can be revoked in bulk via its lockdown function,
// which zeroes out one or more token-and-spender pairs in a single call
```

```
# Point the user at a verified approvals dashboard; never ask them to paste
# a seed phrase or private key into any such tool
open "https://revoke.cash/address/0xYourWalletAddress"
```

6.2 Recovery Flows and Support

The recovery path is where drainer operators concentrate social-engineering effort, because a user who has already lost funds, or fears they are about to, is primed to act fast and skip verification. A fake “support agent” in a Discord or Telegram channel that mimics an official server is a standing pattern across the industry and is the mechanism behind a large share of reported thefts, since no legitimate recovery flow ever needs a seed phrase, a private key, or a signature framed as “verification.” Score this dimension on whether the product publishes an authoritative, hard-to-spoof list of official channels, whether its own staff are trained and contractually barred from requesting secrets, and whether any self-custody recovery mechanism it offers (social recovery guardians, for instance) is resistant to a single compromised guardian.

Recovery mechanism	Impersonation risk	Recommended safeguard
Live chat, Discord, or Telegram “support”	High: attackers impersonate support and request a seed phrase or a disguised signature	Never request a seed phrase or a signature for support purposes; publish an allowlist of official channels in-app
Social recovery guardians	Medium: guardian collusion or a phished guardian key	Time-locked recovery with notification to the original owner; diverse guardian custody
Hardware-wallet reset from a physical backup	Low: requires physical possession of the seed phrase	Educate users to store the phrase offline only, never digitally

Recovery mechanism	Impersonation risk	Recommended safeguard
Exchange-custodied account recovery	Medium: SIM-swap (mobile carrier account takeover) or linked-email takeover	Hardware-key multi-factor authentication (MFA); withdrawal address allowlisting

6.3 Scoring Criteria and Examples

Level	Criteria	Example
0	No in-app visibility into granted approvals; support channels are unverified or accept direct messages	Users must discover and trust an unlinked third-party tool unaided
1	Approvals are visible but revocation requires leaving the app; support policy exists but is not enforced or published	An approvals list exists in account settings, several menus deep
2	In-app revoke available within the primary flow; published official-channel allowlist	One-tap revoke from a notifications or activity screen
3	In-app revoke plus proactive expiry nudges for stale allowances; recovery flows resistant to single-point impersonation	Product flags allowances unused for 90+ days and prompts revocation

7. Dimension: Consistency and Predictability

Consistency and Predictability asks whether the same action always looks and behaves the same way, because an interface that varies its own presentation gives an attacker cover: a spoofed dialog only needs to be as convincing as the legitimate one's own inconsistency allows.

7.1 Terminology and Visual Consistency

Users build a mental model of what a security-relevant dialog looks like from repeated exposure to the genuine one. Every point where the product's own copy, iconography, or color varies for the identical underlying action widens the gap a convincing fake has to clear. This applies within a single platform (a web app and its own settings page using different verbs for the same grant) and across platforms (the same action worded differently on iOS, Android, and the browser extension).

Inconsistent	Consistent
"Approve" on the web app, "Allow" on iOS, "Enable" inside a modal, all for the same allowance grant	"Approve," the same icon, and the same color used everywhere the action appears

Inconsistent	Consistent
"Revoke," "Cancel," and "Remove" used interchangeably for canceling an allowance, including in support documentation	One verb, "Revoke," used consistently across the product and its documentation

7.2 Predictable Navigation and Outcomes

Predictability extends beyond wording to sequence. A confirm button that leads to different outcomes depending on hidden state (network congestion, a stale price quote, a background app update) trains users to click through without reading, which is precisely the habit a phishing clone relies on. Score this criterion against a short checklist: does the same action always require the same number of steps; does using the back button ever silently resubmit or duplicate an action; does a session timeout mid-flow return the user to a safe state rather than an ambiguous one; and does the product avoid unannounced UI changes (a redesigned confirm screen shipped without in-app notice) that retrain users to trust an unfamiliar layout.

- Same action, same step count, every time it is available in the product.
- No silent resubmission on back-button or browser-refresh navigation.
- Session timeout returns to a safe, unambiguous state, not a partially completed one.
- Material UI changes to security-relevant screens are announced in-app before they ship.

7.3 Scoring Criteria and Examples

Level	Criteria	Example
0	Terminology and layout vary noticeably across platforms or contexts for the same action	Approval flow looks and reads differently on mobile versus web with no shared design system
1	Mostly consistent, with a few known divergences	Minor label differences that a style-guide audit would catch
2	Consistent terminology, iconography, and step count across the product's own surfaces	Shared component library enforces identical confirmation screens
3	Consistency verified across platforms and versions with regression testing; changes are announced before shipping	Visual regression tests specifically cover security-critical confirmation screens

8. Dimension: Warnings and Error Handling

Warnings and Error Handling asks whether the interface's alarm bells are calibrated to actual danger, and whether a failure leaves the user knowing what to do next.

8.1 Clear Warnings for Risky Operations

A warning that fires for every transaction trains users to dismiss warnings; a warning that never fires leaves them with no signal at all. The calibrated middle ground scales visual and interactive friction to the underlying risk signal, typically produced by a pre-signing simulation that checks the destination against known-malicious address lists, flags unlimited or unusually large approvals, and flags interaction with new or unverified contracts. Several major wallets, including MetaMask, Coinbase Wallet, Rainbow, and Ledger, integrate third-party risk engines such as Blockaid for exactly this purpose, scanning transactions, addresses, and dApps in real time before a signature is requested (Blockaid).

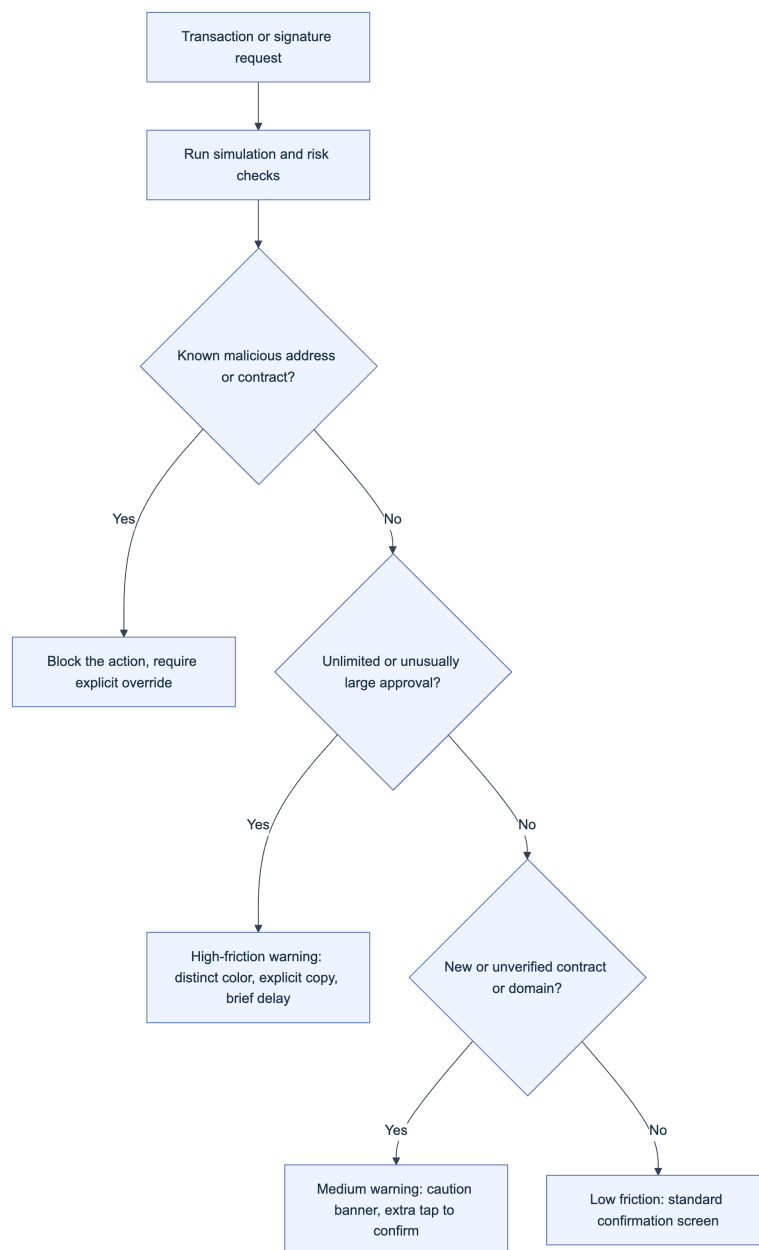


Figure 3. Warning severity scaled to risk signal. A single fixed warning style for every transaction either desensitizes users to real danger or adds needless friction to safe, routine actions.

8.2 Error Messages and Recovery Guidance

An error that names the problem and the fix keeps a user safe; an error that names neither pushes them toward blind retries, which is a pattern drainer operators exploit by presenting a “fixed” retry screen that carries a different, malicious payload. Surface the actual revert reason in plain language, state the specific missing precondition (insufficient balance, expired quote, slippage exceeded), and give a concrete next step rather than a bare “try again.”

Poor	Improved
“Transaction failed.”	“This swap would fail: your USDC balance (\$420) is less than the amount plus network fee (\$431). Add funds or reduce the swap amount.”
“Error 0x4e487b71 (0x11)”	“This transaction would revert due to an arithmetic overflow in the amount field. Try a smaller amount.”

8.3 Scoring Criteria and Examples

Level	Criteria	Example
0	No risk-based warnings; errors are raw codes or generic failure text	A revert surfaces only as an unlabeled hex error code
1	Warnings exist but do not scale with risk; errors name the problem but not the fix	Every transaction shows the same caution icon regardless of actual risk
2	Warning friction scales with risk signal; errors explain cause and next step for common failures	Simulation-based warning for unlimited approvals; balance errors state the shortfall
3	Warnings block or require explicit override for known-malicious targets; errors cover edge cases with actionable guidance	Known drainer contract addresses are blocked outright, not merely flagged

9. Additional Dimensions (Expandable)

The five core dimensions cover the failure modes most directly tied to fund loss. Two further dimensions extend the matrix where a product’s audience or maturity makes them material, and a final section ties every dimension together into one reportable number.

9.1 Accessibility and Inclusivity

A security control that only a sighted, dexterous, English-reading user can act on is not a universal control, it is a control with an exclusion built in, and that exclusion is itself a security gap for every user it leaves out. The Web Content Accessibility Guidelines (WCAG), the W3C’s standard for accessible digital content, define this systematically across four principles:

perceivable, operable, understandable, and robust, at three conformance levels, A, AA, and AAA (WCAG 2.2). Apply WCAG's principles specifically to security-relevant content, not only to general navigation.

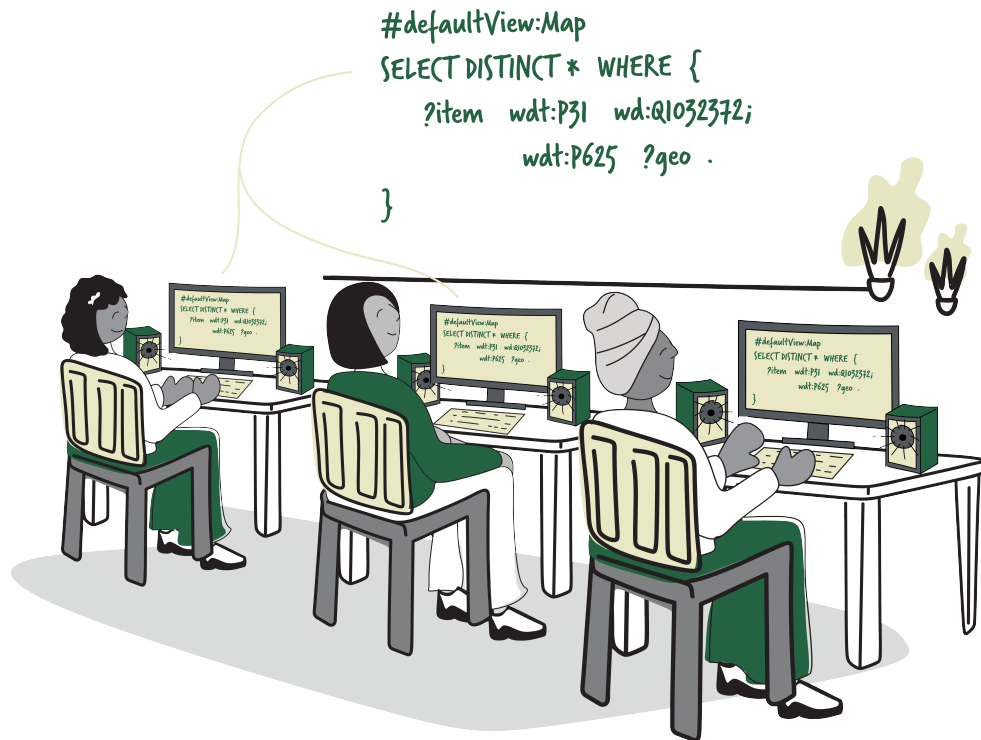


Figure. Developers using screen readers, the same assistive technology this dimension asks a wallet or dApp to support for security-critical fields like a full checksummed address or an unlimited-allowance warning, not only for general page content. Source: [Wikimedia Commons](#), CC BY-SA 3.0.

 Function not allowed, please type password manually.

Password

Figure. Blocking paste is a security anti-pattern as well as an accessibility failure: it prevents password-manager use and forces users to transcribe credentials, encouraging weaker secrets and increasing input error. Source: [W3C WAI, Understanding Accessible Authentication \(Minimum\)](#), W3C document visual used with attribution.

Two-factor Authentication

Enter the code generated by your authentication app

Figure. A split one-time-password control can break paste, autofill, focus order, and screen-reader announcements. Prefer one semantic field with platform autocomplete metadata, while styling digits visually without fragmenting the input model. Source: *W3C WAI, Understanding Accessible Authentication (Minimum)*, W3C document visual used with attribution.

Password

👁

••••••••••••••••

Figure. A correctly labeled show-password control reduces cognitive load and transcription errors without removing masking by default. Its accessible name and pressed state must be exposed to assistive technology, and revealing the value should never be forced. Source: *W3C WAI, Understanding Accessible Authentication (Minimum)*, W3C document visual used with attribution.

- Full checksummed addresses are announced by screen readers, not just the truncated visual string, via an explicit `aria-label`.
- Risk level is never conveyed by color alone (WCAG 2.2, success criterion 1.4.1); pair color with an icon and a text label.
- Confirmation screens remain usable at 200% text zoom without clipping the confirm or cancel control (WCAG 2.2, success criterion 1.4.4).
- Any video-based security education includes captions and a text transcript.
- A complete keyboard-only path exists to review and revoke an approval, with no mouse required.

9.2 Onboarding and Education

Security education that happens once, at signup, is forgotten by the time it matters. The pattern that scores better is just-in-time: the explainer for “what is an unlimited allowance” appears the first time the user is actually asked to grant one, not buried in a one-time welcome tour they clicked through. Pair contextual explainers with a persistent, inline glossary for terms like allowance, gas, and blind signing (signing a request without being able to verify what it actually

authorizes), and consider a periodic re-education nudge after long account dormancy, since a returning user after months away is a common target for social engineering that exploits their rustiness with the product's real flows.

- Contextual explainer triggered at the first unlimited-approval prompt, not only in a one-time onboarding wall.
- Inline glossary linked from any screen using a term like allowance, gas, or blind signing.
- Optional sandbox or testnet practice transaction before a first-time high-value mainnet action.
- Re-engagement nudge on long dormancy, framed as a security refresher rather than a marketing prompt.

9.3 Aggregate Score and Reporting

Report per-dimension scores alongside, never instead of, the aggregate, because averaging hides a single catastrophic zero behind four healthy threes. Weight the five core dimensions by how directly each one gates fund loss, and treat the two expandable dimensions as separate line items unless the product's own risk profile (a large low-vision user base, for instance) justifies folding them into the core weight.

Dimension	Weight	Rationale
Transaction and Action Clarity	1.0	Root cause of most reported drainer losses
Confirmation and Consent	1.0	The last control before an irreversible signature
Revocability and Recovery	1.0	Determines how long a single mistake stays exploitable
Consistency and Predictability	0.75	Reduces the odds a spoofed screen goes unnoticed, but rarely the direct cause of loss
Warnings and Error Handling	1.0	Directly gates whether calculated risk ever reaches the user
Accessibility and Inclusivity (expandable)	0.5	A control that fails silently for the users it excludes
Onboarding and Education (expandable)	0.5	Reduces the likelihood, not the severity, of a given failure

Compute the core aggregate as the sum of weighted core-dimension scores divided by the maximum possible weighted total, expressed as a percentage: $(\sum \text{weight} \times \text{score}) / (3 \times \sum \text{weight}) \times 100$. Report the expandable dimensions alongside as a secondary line rather than folding them silently into the headline number, so a strong core score never masks a product that is inaccessible or teaches nothing. Re-run the full matrix at three points in the product lifecycle at

minimum: design review, before every release that touches a signing or approval flow, and on a fixed quarterly cadence regardless of release activity, since a dimension can regress silently when an unrelated redesign changes shared components.

Key controls for Part 2

- Score every signing-relevant flow on all five core dimensions before release; treat a 0 or 1 on Confirmation and Consent or Warnings and Error Handling as a release blocker.
- Default token approvals to an exact, time-bound grant (Permit2-style) rather than an unlimited, indefinite one.
- Give every product its own in-app approvals dashboard and one-tap revoke, not only a link to a third-party tool.
- Scale warning friction to simulation-derived risk signal instead of applying one fixed warning style to every transaction.
- Keep terminology, iconography, and step count identical for the same action across every platform the product ships on.
- Treat accessibility and onboarding as scored dimensions, not optional polish, when the product's audience or risk profile makes them material.

Part III: Best Practices

The UX Security Scoring Matrix in Part II gives you dimensions to grade an interface against: clarity, confirmation, revocability, consistency, and error handling. This part turns each dimension into shippable patterns, working through the surfaces a Web3 user actually touches: the wallet's signing screen, the handshake that connects a wallet to a decentralized application (dApp), the allowance a token grants a spender, the alert that fires when something goes wrong, and the first minutes a new user spends learning the product.

10. Wallet and Signing UX

The wallet is the last checkpoint before an on-chain action becomes irreversible, and every pattern in this chapter answers the same question: does the interface show the user what will actually happen, in language and on hardware the user can trust, before the signature is produced.

10.1 Custodial vs. Non-Custodial and Cold vs. Hot: UX Implications

Two independent axes shape a wallet's security posture and UX budget. The first is custody: a custodial wallet (an exchange account, a managed multi-party computation (MPC) service) holds keys on the user's behalf and offers familiar patterns like password reset, trading self-custody risk for counterparty risk. A non-custodial wallet puts the user in sole possession of key material, usually a seed phrase or a hardware secure element, and inherits none of the operator's recovery infrastructure: lose the phrase and the funds are unrecoverable, a UX burden custodial products do not carry. The second axis is connectivity: a hot wallet (browser extension, mobile app) is reachable by any page or process the device runs, making it the default target for clipboard hijackers and drainer kits. A cold wallet (hardware device, air-gapped signer) keeps the private key on a device with no persistent network path, trading a few extra seconds of friction for immunity to remote key exfiltration.

Neither axis has one correct answer; the right choice tracks the value at risk.

Actor profile	Recommended class	Rationale
Retail, daily spending balance	Non-custodial hot	Convenience outweighs the smaller balance at risk; pair with allowance hygiene (12.1)
Retail, long-term holdings	Non-custodial cold	Infrequent use tolerates the friction; removes remote-exfiltration risk
DAO or protocol treasury	Non-custodial cold, multisig	No single signer should authorize alone; on-device verification (10.2) mandatory
Institutional custody	Custodial MPC or qualified custodian	Regulatory and insurance requirements usually outweigh counterparty risk

Surface this tradeoff explicitly rather than defaulting silently to whichever option is easiest to build; the choice a user makes at onboarding is rarely revisited.

10.2 Hardware Wallet On-Device Verification for Critical Actions

A hardware wallet's security promise rests on one property: the screen the user reads and the key that signs live on the same tamper-resistant device, so a compromised host computer cannot alter what is confirmed. That promise breaks the moment a critical action is approved by glancing at the connected computer's screen instead of the device's own display, because the host is exactly the component an attacker is most likely to control. The February 2025 Bybit theft, in which North Korea's Lazarus Group (tracked by the FBI as "TraderTraitor") stole approximately \$1.5 billion by manipulating the transaction shown to Safe multisig cold-wallet signers ([FBI Internet Crime Complaint Center \(IC3\) flash alert, 26 Feb 2025](#)), is the reference case: the front end displayed a routine transfer while the actual signed payload changed control of the wallet, and because it was never decoded into human-readable form on the device itself, the on-device screen could not catch what the browser hid.

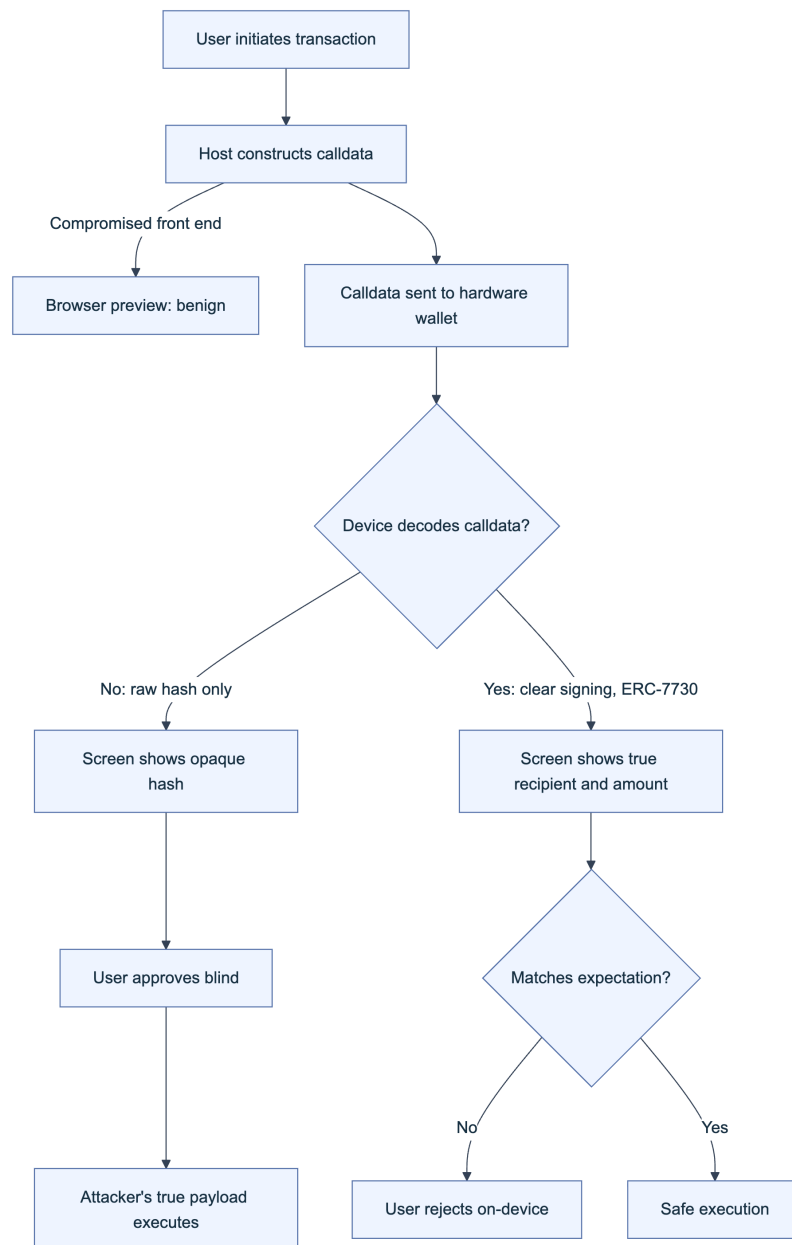


Figure 1. On-device decoding is the only checkpoint a compromised host cannot spoof. Without it, the hardware screen is a blind rubber stamp.

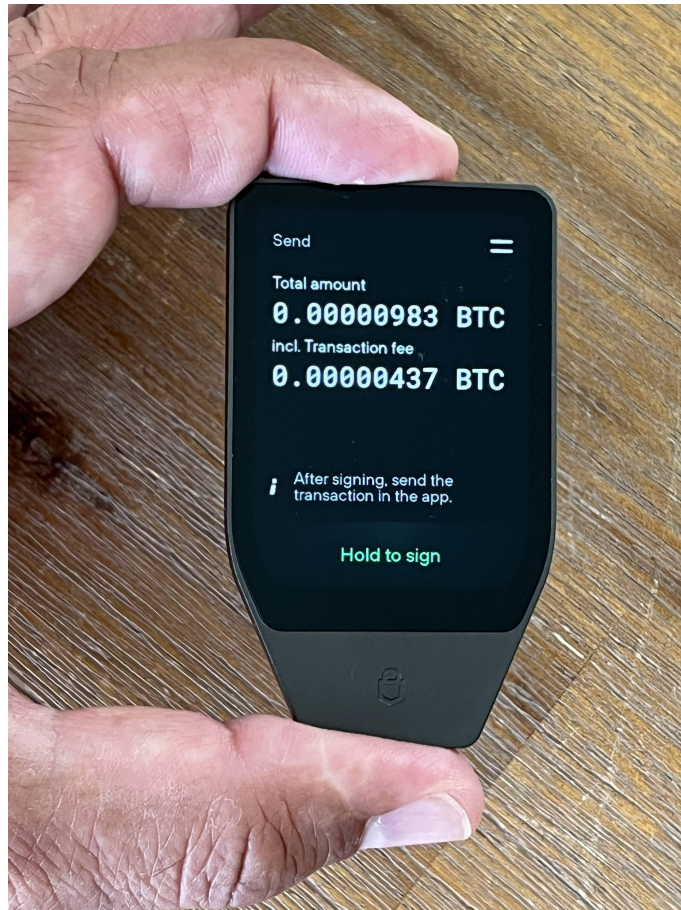


Figure. A hardware wallet's on-device screen, the checkpoint this section describes: the device decodes and displays what it will actually sign, so a compromised host cannot substitute a different transaction without the mismatch being visible here. Source: [Wikimedia Commons](#), CC0 1.0 (public domain).

The industry response is the Clear Signing initiative Ledger began in 2023, now the Ethereum Foundation-backed Clear Signing Alliance, built on the open **ERC-7730** metadata standard, which lets a hardware device translate raw calldata into “the recipient, the amount, the type of approval, and which dApp is requesting it, in simple language” on a certified, tamper-proof display ([Ledger, Clear Signing](#)). Treat these as critical actions that must never be approved from a raw hash: any high-value transfer, any token or non-fungible token (NFT) approval, any change to a multisig owner set or threshold, any contract upgrade or ownership transfer, and any EIP-7702 delegation (Section 10.4). Where a target contract has no ERC-7730 descriptor yet, the wallet should say so explicitly (“this action cannot be clear-signed”) rather than falling back silently to blind signing.

10.3 EIP-712 and Human-Readable Signature Review

Before structured signing existed, a wallet had only one way to ask a user to authorize arbitrary data: show a hex string and a “Sign” button. [EIP-712](#) replaced that with a standard for hashing and signing typed structured data, so a message can be rendered the way a Solidity struct is: named fields, explicit types, and a domain separator (contract name, version, chain ID, and verifying contract address) that stops the same signature from being replayed against a different

application or chain. A wallet that supports `eth_signTypedData_v4` can walk the struct and print `spender: 0xA1b2...`, `value: 500 USDC`, `deadline: 2026-08-17 14:00 UTC` instead of a 66-byte blob, which is the difference between a user who can catch a mismatched spender address and one who cannot.

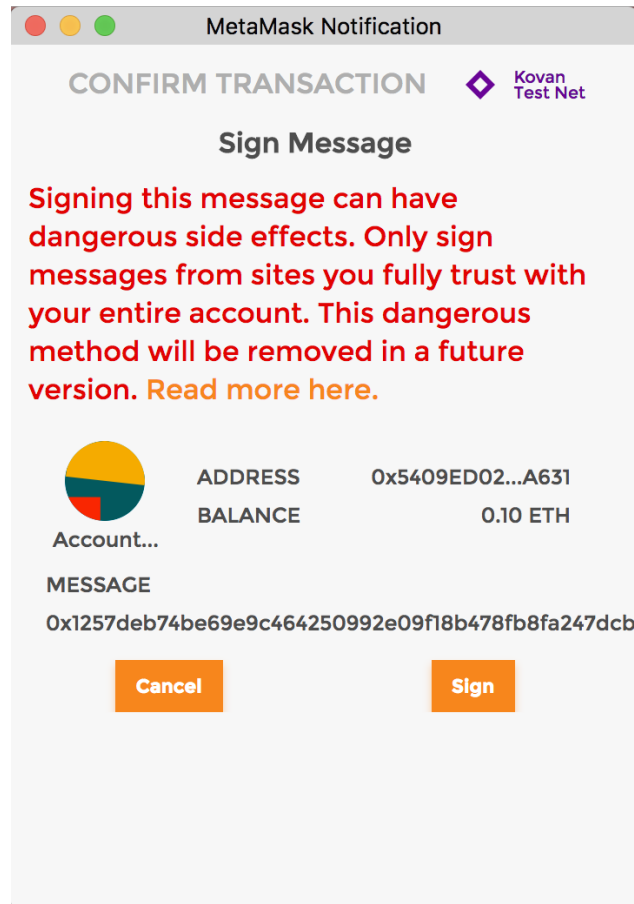


Figure. The failure state EIP-712 was designed to replace: an `eth_sign` prompt exposes bytes but no domain, field names, recipient, amount, or deadline that a human can verify. Source: *EIP-712 specification*, CCO.

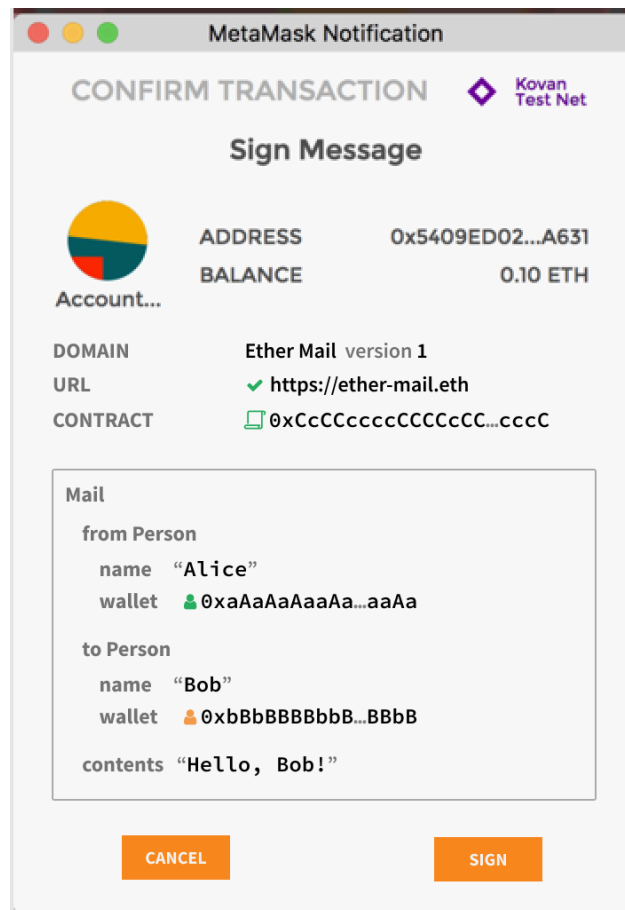


Figure. The same authorization class represented as typed structured data. Named fields are necessary but not sufficient: the wallet must still render the domain separator, verifying contract, chain, spender, value, nonce, and expiry in a layout that makes mismatches salient. Source: [EIP-712 specification](#), CC0.

```
{
  "domain": { "name": "USD Coin", "version": "2", "chainId": 1, "verifyingContract":
    "0xA0b8...eB48" },
  "types": {
    "Permit": [
      { "name": "owner", "type": "address" },
      { "name": "spender", "type": "address" },
      { "name": "value", "type": "uint256" },
      { "name": "nonce", "type": "uint256" },
      { "name": "deadline", "type": "uint256" }
    ]
  },
  "primaryType": "Permit",
  "message": { "owner": "0xUser...", "spender": "0xRouter...", "value": "500000000",
    "nonce": 4, "deadline": 1755436800 }
}
```

Two practices follow. dApp developers should never request a legacy `eth_sign` or unstructured `personal_sign` over arbitrary bytes when a typed-data equivalent exists; most major wallets, including MetaMask, now warn or block `eth_sign` by default precisely because it lets a malicious dApp request a signature over anything, including a disguised transaction hash. Wallet developers

should treat an unrecognized message type the way they treat undecodable calldata in Section 10.2: flag it, name the risk in plain language, and require an extra confirmation step. This directly satisfies the **Transaction and Action Clarity dimension** of the scoring matrix in Part II.

10.4 EIP-7702 and Smart-Contract Features for EOAs: New Risks

EIP-7702, activated on Ethereum mainnet as part of the Pectra upgrade on 7 May 2025 ([Ethereum Foundation](#), [Pectra roadmap](#)), lets an externally owned account (EOA) attach executable code to itself for the duration of a transaction by including a signed authorization tuple, enabling call batching, sponsored (gas-paid-by-a-third-party) transactions, and privilege de-escalation through sub-keys. It is a genuine UX improvement and a genuinely new class of risk, because the EOA's authorization signature is a delegation of authority, not a one-time action. The EIP's own security considerations state that “a poorly implemented delegate can allow a malicious actor to take near complete control over a signer's EOA,” and warn that delegate contracts must independently enforce replay protection, value limits, gas limits, and target restrictions ([EIP-7702 Security Considerations](#)). The upgrade also breaks the long-standing invariant that `tx.origin == msg.sender` holds only at the top of the call stack, quietly invalidating some existing reentrancy and sandwich-attack guards.

The OWASP Web3 Attack Vectors Top 15 already documents scam pages that frame a 7702 authorization as a routine “upgrade your wallet” prompt, deceiving users into delegating their EOA's code to an adversarial contract that sweeps its assets ([OWASP Web3 Attack Vectors](#), **WA06**).

Risk	UX mitigation
Malicious delegate takes full control of the EOA	Treat any 7702 authorization request with the same critical-action UI as an owner change (10.2): full on-device address display, no batching with unrelated actions
Generic “upgrade your wallet” framing hides the delegation's scope	Show the exact delegate contract address and, where an ERC-7730 descriptor exists, what functions it exposes; never render a vague label
Front-run initialization of a newly delegated account	Require the EOA's own signature (<code>ecrecover</code>) for setup parameters; never accept unsigned initialization from an observer
Sponsor (relayer) risk of asset sweep before reimbursement	dApps offering sponsored transactions should disclose the sponsor's bonding or reputation mechanism in-product

Never let a dApp present a 7702 signature request as anything less than “you are granting a smart contract permanent code-level control over this wallet address until you revoke it.”

10.5 Account Abstraction Wallets: Verification and Consent UX

ERC-4337 account abstraction (AA) moves the wallet from a fixed protocol-level EOA to a smart contract account with programmable validation logic, using a `UserOperation` object that a Bundler packages into a call against a shared EntryPoint contract, optionally paid for by a Paymaster. This breaks the “one click, one signature” model Sections 10.2 and 10.3 assume: a single UserOperation can bundle several calls, gas can be sponsored so the user never sees a native-token cost, and under ERC-7579 modular accounts, a session key can be granted to a dApp or automation service to act without any further per-transaction confirmation, constrained by spending limits, time windows, and target restrictions enforced by a session-management module.

That programmability is a UX win only if consent UX keeps pace. A wallet’s decoding obligation from Section 10.3 extends to every call inside a UserOperation, not just the top-level intent, because a bundled operation can hide an unrelated approval inside an otherwise-expected swap; show a net asset-flow summary (what leaves, what arrives, what allowances change) rather than a generic “confirm transaction” button. For session keys, provide a persistent panel listing every active grant with its scope, spend limit, expiry, and an instant, on-chain-enforceable revoke action, the AA equivalent of the allowance dashboard in Section 12.1. Where the account has no hardware secure element, pair it with a WebAuthn passkey or hardware co-signer so the on-device verification guarantee from Section 10.2 is not silently dropped.

11. dApp and Connection UX

Before a wallet signs anything, it has to trust that it is talking to the dApp the user intended to reach. This chapter covers the two controls that anchor that trust: verifying the connecting domain, and never letting the user discover a dApp through an unverified link.

11.1 WalletConnect Verify API and Domain Verification

Browser-extension wallets get an origin check for free: the browser enforces that a page can only request signatures for the origin it is served from. WalletConnect-style connections, where a desktop dApp displays a QR code or deep link and a separate mobile wallet scans or receives it, have no such built-in check, because the wallet never loads the dApp’s page and cannot inspect its origin directly. The Verify API, offered by WalletConnect’s parent project Reown, closes that gap by having the relay attach a verification attestation to the session proposal: it compares the domain the dApp declared in its metadata against the domain actually serving the connection request and cross-references both against a registry of known-malicious domains, then returns a status the wallet renders as a badge before the user approves the pairing (see [Reown documentation](#) for current implementation details).

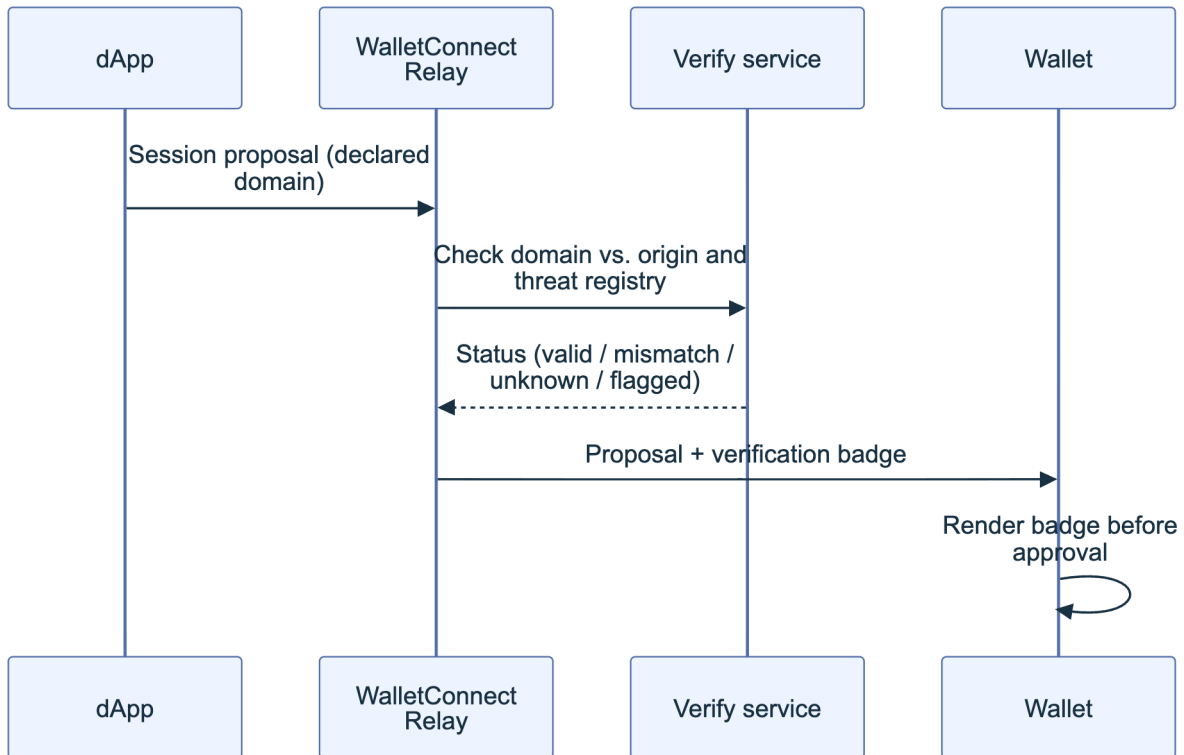


Figure 2. The Verify API attaches an out-of-band domain check to a connection that has no native browser origin boundary to rely on.

Badge state	Meaning	Recommended wallet UX
Verified	Declared domain matches serving origin, not flagged	Green indicator, standard flow
Domain mismatch	Declared domain differs from actual serving origin	Prominent warning; require an extra confirmation tap
Unknown	dApp has not registered verifiable metadata	Neutral indicator, but still surfaced; silence is not safety
Flagged	Domain matches a known-scam entry	Block by default; require an explicit, labeled override

The control only works if wallets refuse to render every state identically; a badge nobody sees is not a control.

11.2 Bookmarks and Verified Entry Points

The single most exploited human factor in Web3 phishing is not a cryptographic weakness, it is the address bar. Punycode homograph domains, malicious search-engine ads that outrank the legitimate result for “Uniswap” or “MetaMask,” and compromised social-media links all attack the same moment: the instant a user chooses which URL to trust. The fix is procedural, not technical. Instruct users to type a canonical URL once, verify it against the project’s official channels, and bookmark it rather than searching or clicking a link on every subsequent visit; a bookmark cannot

be typosquatted and removes the search-ad attack surface entirely. Wallet vendors reinforce this by curating vetted dApp directories inside their own apps (hardware-wallet companion apps and mobile “discover” tabs vet an origin before listing it), and extensions such as MetaMask ship a community-maintained phishing-domain blocklist that intercepts known-bad hosts before the page renders.

Checklist for teams publishing an official entry point: - Register and defend look-alike and homograph variants of your primary domain. - Publish the canonical URL prominently and consistently across every official channel so users have one authoritative source to compare against. - Apply the DNS-hardening controls in the DNS and Hosting Security Handbook (02), including registrar lock and DNSSEC, so the canonical domain itself cannot be hijacked. - Submit your domain to major wallet phishing-blocklist maintainers and the WalletConnect Verify registry, and never rely on a paid search ad as a distribution channel.

12. Approval and Allowance UX

An approval is a standing authorization, not a one-time action, and the gap between what a user intends to grant and what the interface actually requests is one of the most exploited surfaces in Web3.

12.1 Granular and Time-Limited Approvals; Revocation Flows

The ERC-20 `approve` function takes an arbitrary allowance amount, and the path of least resistance for a dApp is to request `type(uint256).max`, an unlimited allowance, so the user never has to re-approve later. That convenience is exactly the mechanism behind approval-phishing drains: once a spender is authorized for an unlimited amount, one malicious `transferFrom` call, at any future point, empties the balance with no further signature required. Chainalysis attributed at least \$374 million in 2023 losses to this pattern, tracked by OWASP as its own Web3 attack-vector category, WA06 ([OWASP Web3 Attack Vectors Top 15](#)).

```
// Requested by the dApp, decided by the interface, not the protocol
IERC20(token).approve(spender, 500e6);           // exact amount: expires with the
trade
IERC20(token).approve(spender, type(uint256).max); // unlimited: standing risk until
revoked
```

Uniswap’s `Permit2` improves on raw `approve` by centralizing allowances in one audited contract and letting a spender request a signature-based, time-bound permit instead of a separate on-chain transaction per integration. That is a real improvement, but it moves the risk rather than eliminating it: a permit is a signature, not a transaction, so it produces no separate on-chain

approval event for a monitoring tool to flag, and demands the exact same EIP-712 clear-signing rigor from Section 10.3, arguably more, since there is no second confirmation step to catch what the first missed.

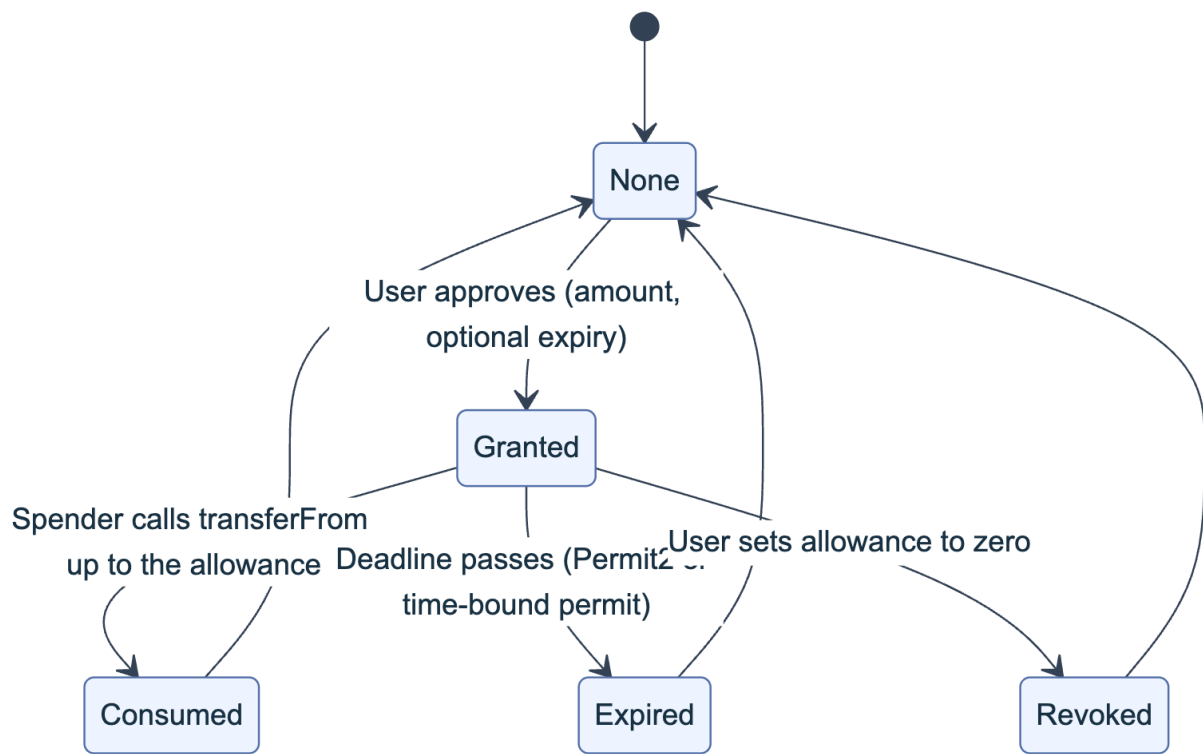


Figure 3. An allowance is standing risk between **Granted** and any exit state. The UX goal is to minimize time spent in **Granted** and make every exit reachable in one action.

Concrete UX rules for any interface that requests an allowance:

Rule	Why it matters
Default the requested amount to the exact transaction value, not unlimited	Removes the standing-risk window for the common case
Gate unlimited or custom amounts behind an explicit expand action with a plain-language warning	Satisfies the Warnings and Error Handling dimension ; unlimited becomes deliberate, not default
Default a bounded expiry for signature-based permits	Converts standing risk into a self-expiring one, even without manual revocation
Ship or link an allowance dashboard (in-product or Revoke.cash / block-explorer checkers)	Hardware wallets give no protection against an already-granted approval; users cannot manage what they cannot see
Make revocation one click from that dashboard	Satisfies the Revocability and Recovery dimension directly

13. Notification and Alert UX

Detection only has value if it reaches the user before or immediately after damage occurs, and that alert path is its own discipline, distinct from the signing-time controls in Chapter 10. Two moments matter. Pre-signature, transaction-simulation tooling independent of the dApp's own front end can execute the pending transaction against current chain state and show the predicted asset-flow diff before the user signs, catching a manipulated preview exactly like the one in the Bybit incident (Section 10.2), because the simulation reads the real calldata, not whatever the compromised page chose to render. Post-broadcast, monitoring services should watch for the small set of events that precede most losses (a new unlimited approval, an unusually large outbound transfer, a multisig owner-set change, an EIP-7702 delegation) and push that alert through a channel the user actually checks, not only an in-app inbox that may go unread for days.

The central design failure here is alert fatigue: research on browser and phishing warnings, going back to Sunshine, Egelman, Almuhiemedi, Atri, and Cranor's [Crying Wolf: An Empirical Study of SSL Warning Effectiveness](#) (USENIX Security 2009), shows that users warned too often, or warned generically, learn to click through without reading. Apply severity tiers rather than a single warning style, and let friction scale with severity so the interface never trains a user to dismiss the exact alert that matters most.

Channel	Latency	Guarantee	Best suited for
Pre-sign simulation warning	Before signature	Blocks or requires override before the action is possible	Every transaction and typed-data signature, especially unrecognized calldata
In-wallet post-broadcast alert	Seconds to minutes	Informational; damage may already be irreversible	New approvals, large transfers, ownership changes
Push notification / SMS	Seconds to minutes	Reaches the user outside the app	High-severity events, app not open
Email digest	Hours	Low urgency, high recall for review	Routine activity summaries, not time-critical incidents

Tie the highest severity tier (large approval to a new spender, ownership transfer, unrecognized delegate contract) to the mandatory-confirmation pattern from Section 10.2 rather than a dismissible toast, and route confirmed incidents into the escalation process defined in the Incident Response Handbook (06).

14. Onboarding and Education

The first minutes a user spends in a Web3 product carry a contradiction: the interface must convey genuinely high-stakes information (an unrecoverable seed phrase, an irreversible transaction model) without producing so much friction or fear that the user abandons setup. A wall of security disclaimers read once at signup is measurably ineffective, because the concepts stay abstract until they become concrete, so the practice that performs better is progressive disclosure: teach the minimum needed for the current step and defer the rest until it is directly relevant.

Your Seed Phrase

Your Seed Phrase is used to generate and recover your account.

1. issue	2. flame	3. sample
4. lyrics	5. find	6. vault
7. announce	8. banner	9. cute
10. damage	11. civil	12. goat

Please save these 12 words on a piece of paper. The order is important. This seed will allow you to recover your account.

☒ I understand that if I lose my seed phrase that I will not be able to recover my account.

Accept

Figure. The seed-phrase creation screen this section discusses: the moment a wallet must convey an unrecoverable, high-stakes consequence without stalling setup, which is why the recommended pattern pairs a concrete warning with an active recall step rather than a passive checkbox. Source: [Wikimedia Commons](#), CC BY-SA 4.0.

Concretely, that means introducing seed-phrase custody at wallet creation with a short, concrete consequence statement (“if you lose this phrase, no company, including us, can recover your funds”) rather than legal-style caveats, verified with a recall step instead of a checkbox. It means routing a trivial-value first transaction, a testnet or low-value transfer, before any flow that requests a meaningful approval, so the user’s first exposure to the signing screen (Section 10.3) is low-stakes. It means triggering just-in-time explainers at first exposure to a specific risk rather than front-loading everything: the first blind-signing prompt, unlimited-approval request, and EIP-7702 delegation request should each surface a short explanation the moment they occur,

when motivation to read it is highest. Some teams go further with sandboxed simulated-phishing exercises, a fake urgent prompt in a labeled practice environment, so a user learns to recognize social-engineering pressure without real funds at risk.

Checklist for an onboarding flow: - Defer non-essential security content until it becomes actionable; do not front-load one long disclosure screen. - Verify seed-phrase capture with an active recall step, not a passive “I have saved my phrase” checkbox. - Route the first meaningful transaction through a low-value or testnet path before requesting a high-value approval. - Bind every first exposure to a named risk category (blind signing, unlimited approval, delegation) to a short contextual explainer, reusable across the notification tiers in Chapter 13. - Measure drop-off at each checkpoint and treat a spike as a UX defect to fix, not a security requirement to relax.

Key controls for Part 3

- Decode every signature request into human-readable form (EIP-712, ERC-7730 clear signing) with on-device confirmation for critical actions; never let a raw hash stand in for a reviewable transaction.
- Treat EIP-7702 delegation and account-abstraction session-key grants as owner-level actions with full address disclosure, never a generic “upgrade” prompt.
- Verify the dApp’s domain out of band (WalletConnect Verify API) and route users to it only through bookmarked or vetted entry points, never a search ad.
- Default token and permit approvals to the exact amount needed with a bounded expiry, backed by a one-click revocation dashboard.
- Tier alerts by severity so friction scales with risk, and reserve just-in-time education for the moment a specific risk first appears.

Part IV: Implementation and Evaluation

Part IV turns the scoring matrix from a reference document into a working control instead of a document teams consult once at launch and never again. It covers where UX security review gates sit in a normal software development lifecycle (SDLC), how to run a matrix-based audit and track its findings to closure, and how usability testing and incident feedback keep the matrix honest as wallets, standards, and attackers change. A dimension defined in Part II only protects users if it is re-checked on every release that touches a signing or approval flow, not just the first one.

15. Integrating UX Security into Development

UX security fails most often not because a team disagrees with the matrix, but because nobody owns the moment where a design either passes through a gate or ships without one. This chapter places those gates inside the SDLC, assigns them to roles, and automates the parts a human should not have to remember.

15.1 UX Security in the Software Development Lifecycle (SDLC)

Treat UX security the same way contract security treats a threat model: as an input to design, not an inspection applied after the fact. [NIST SP 800-218, the Secure Software Development Framework \(SSDF\)](#), organizes secure development into four practice groups: Prepare the Organization, Protect Software, Produce Well-Secured Software, and Respond to Vulnerabilities. [OWASP's Software Assurance Maturity Model \(SAMM\)](#) covers similar ground through five business functions: Governance, Design, Implementation, Verification, and Operations. Both frameworks share the same underlying claim: security that only appears at the Verification stage is security applied too late and too narrowly to catch flow-level problems like a missing confirmation step or an approval that silently defaults to unlimited.

Mapped onto a typical feature's path from idea to production, UX security activities look like this:

SDLC phase	UX security activity	Owning artifact
Requirements and discovery	Threat-model the flow against the UX threat categories in Part I, Chapter 2 (phishing, allowance abuse, blind signing, calldata/UI mismatch)	Threat-model worksheet attached to the feature ticket
Design	Score the proposed flow against the matrix dimensions before implementation begins	Pre-build matrix scorecard (Section 16.1)
Implementation	Enforce clear-text calldata rendering, confirmation modals, and revocation affordances as defaults in the component library, not opt-ins a developer must remember	Shared UI component library with security defaults
Verification	Run automated checks and manual QA against the matrix criteria	CI report plus audit checklist (Sections 15.4 and 16.1)
Release	Gate deployment on an aggregate score threshold	Release sign-off record (Section 16.3)
Operations	Monitor support tickets, wallet-side warnings, and incident reports for drift from the scored baseline	Feedback log (Section 17.2)

The practical consequence is that a UX security finding raised at the design phase costs a redrawn mockup, while the same finding raised after launch costs a hotfix, a user-facing incident, and a re-audit. Front-load the checks.

15.2 Design Reviews and Security Requirement Gates

A design review gate is the point where a proposed flow must clear a defined bar before engineering starts building it. Without an explicit gate, “we’ll fix the UX later” becomes the default, and later rarely arrives before the feature ships. Define the gate as a checklist a design cannot pass the review without satisfying, or without an explicit, logged exemption signed by a named security owner.

Design review gate checklist

- ☐ Every state-changing action (transfer, approval, delegation, signature) has an explicit UI element naming the exact action being taken.
- ☐ Recipient, amount, and asset are rendered from decoded calldata, not solely from a developer-supplied label (Part II, Section 4.1).
- ☐ High-risk actions, including unlimited approvals, contract upgrades, ownership transfers, and EIP-7702 delegation grants, require a distinct, higher-friction confirmation step (Part II, Chapter 5).
- ☐ A revocation path exists for any granted permission and is discoverable within two navigation steps of the flow that created it (Part II, Chapter 6).
- ☐ Terminology and iconography match the rest of the product and any prior version of the same flow (Part II, Chapter 7).
- ☐ Warning and error copy has been reviewed by both security and content design, not drafted by engineering alone (Part II, Chapter 8).
- ☐ The design carries a recorded matrix score, or a written exemption with an expiry date, before implementation begins.

Log the outcome of every gate review, pass or fail, in the same tracker used for the feature. A gate that exists only as a verbal conversation in a design review meeting leaves no evidence for the audit in Chapter 16, and no record for the postmortem in Section 17.2 if the flow later causes an incident.

15.3 Cross-Functional Collaboration: Security, Design, and Engineering

UX security sits across three disciplines that rarely report to the same manager, and ambiguity about who decides what is the most common reason a gate gets skipped under deadline pressure. A responsibility matrix resolves that ambiguity before a release, not during one.

Matrix dimension	Product / Design	Frontend Engineering	Security	QA
Clarity	A	R	C	I
Confirmation and consent	C	R	A	I
Revocability and recovery	C	R	A	C
Consistency and predictability	A	R	I	C
Warnings and error handling	C	R	A	C

R = Responsible (does the work), A = Accountable (owns the sign-off), C = Consulted (input sought before the decision), I = Informed (told after the decision).

Security holds accountability for the dimensions most directly tied to fund loss (confirmation, revocability, warnings), so a deadline cannot unilaterally lower that bar. Product and design own clarity and consistency, since both are inseparable from the product's overall voice. Engineering is responsible everywhere: it builds what the other two roles specify. QA tests against the same criteria defined in Chapter 16 rather than inventing its own bar.

15.4 Tooling, Linting, and CI Checks for UX Security Patterns

Manual review does not scale to every pull request, and a human reviewer tires of flagging the same pattern on the fortieth occurrence. Encode the cheapest, highest-confidence checks as automated gates in continuous integration and continuous delivery (CI/CD), so a violation blocks a merge instead of waiting for the next audit cycle.

```
# .github/workflows/ux-security-checks.yml
name: ux-security-checks
on: [pull_request]
jobs:
  matrix-static-checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - name: Verify rendered summary matches decoded calldata
        run: npm run test:calldata-render -- --ci
      - name: Flag unlimited approval calls without a confirmation gate
        run: npx eslint . --rule '{"custom/no-unguarded-set-approval-for-all": "error"}'
      - name: Confirm every state-changing action carries a matrix score annotation
        run: node scripts/check-matrix-annotations.js
```

A lint rule can catch the specific pattern that most often defeats the Confirmation and Consent dimension: an unlimited or wildcard approval issued without a distinct, higher-friction confirmation step wrapping it.

```
// eslint-rule sketch: flag setApprovalForAll / approve(MaxUint256) calls
// that are not routed through the shared confirmHighRiskAction() gate.
module.exports = {
  create(context) {
    return {
      CallExpression(node) {
        const callee = node.callee.property && node.callee.property.name;
        const isUnlimitedApprove =
          callee === 'approve' &&
            /MaxUint256/.test(context.getSourceCode().getText(node.arguments[1]));
        const isSetApprovalForAll = callee === 'setApprovalForAll';
        if ((isUnlimitedApprove || isSetApprovalForAll) && !isWrappedInConfirmationGate(node)) {
          context.report(node, 'Unlimited or blanket approval must route through confirmHighRiskAction()');
        }
      },
    },
  },
};
```

These checks are detection aids, not a substitute for the human review in Section 16.1: they catch known bad patterns cheaply and consistently, while a matrix audit catches the flows a static rule cannot reason about, such as whether the plain-language summary a developer wrote is actually accurate.

16. Review and Audit Using the Matrix

Where Chapter 15 builds the matrix into how a feature gets designed and shipped, this chapter turns it into a repeatable audit discipline, the interface-layer counterpart to the testing rigor the [Smart Contract Security Testing Guide \(SCSTG\)](#) applies to contract code.

16.1 Conducting a Matrix-Based UX Security Audit

An audit is a bounded, evidence-producing walk through a flow, not a general impression of whether the product “feels safe.” Run it as a fixed procedure so two auditors reach comparable scores on the same flow.

1. Scope the audit to a specific feature, flow, or full product surface, and record the scope before starting.
2. Walk the flow start to finish twice: once as a first-time user with no prior context, once as a returning user who has completed the flow before.
3. Score each in-scope dimension (Clarity, Confirmation and Consent, Revocability and Recovery, Consistency and Predictability, Warnings and Error Handling, and any Additional Dimensions in scope) using the levels defined in Part II, Section 3.3.
4. Capture evidence, a screenshot, a decoded calldata dump, or a session recording, for every score below the maximum. A score with no evidence is not reproducible and will not survive a re-audit dispute.
5. Compute the aggregate score per Part II, Section 9.3, and compare it against the release threshold set for that flow’s risk tier.
6. File one remediation ticket per below-threshold dimension, not one ticket per flow, so fixes can be tracked and prioritized independently.

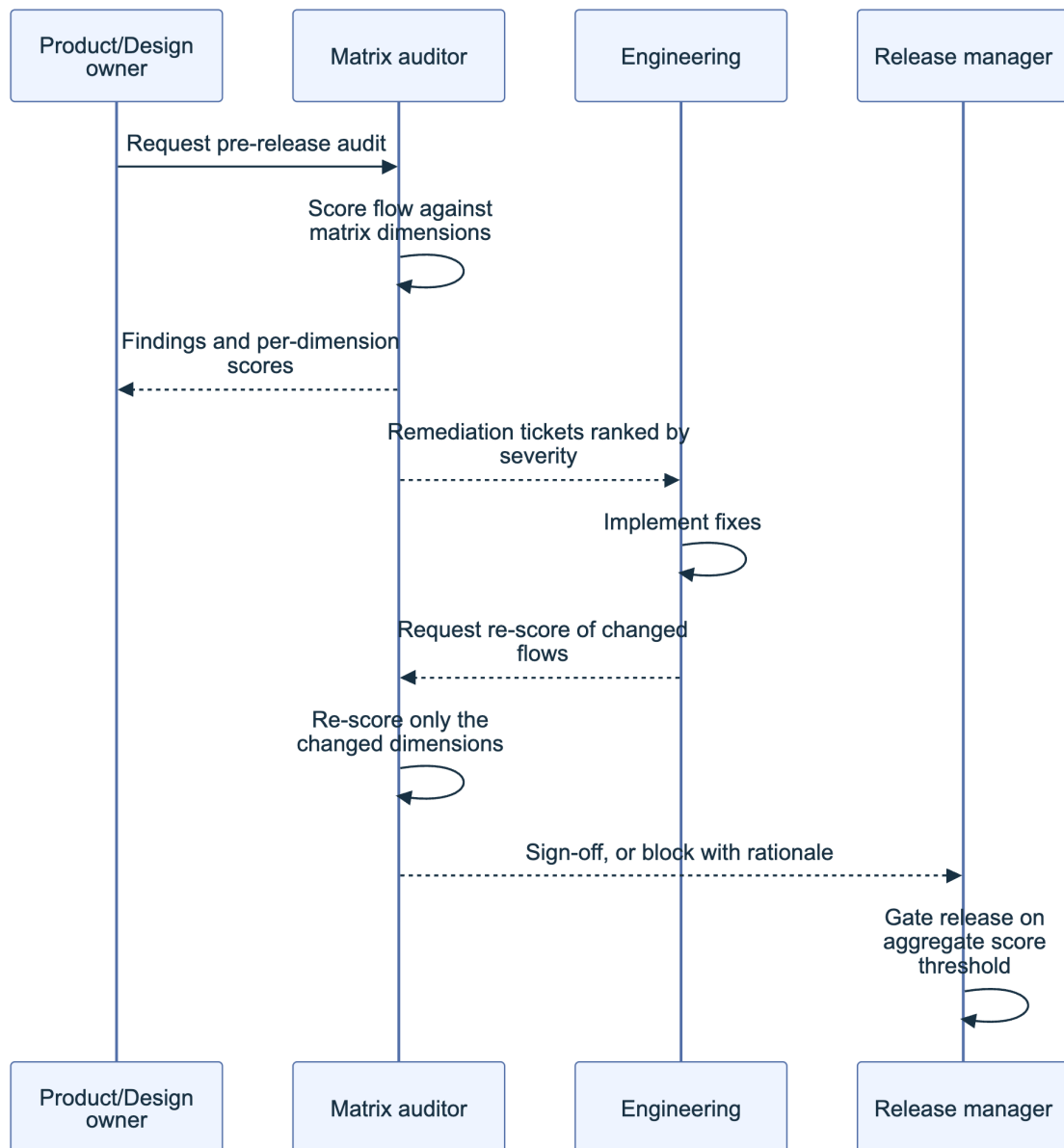


Figure 4.1. The audit-to-release interaction. Re-scoring after remediation is scoped to the changed dimensions, not a full re-walk, so the loop stays fast enough to fit inside a release cycle.

An example scorecard for a token-approval flow makes the procedure concrete:

Dimension	Score (0-3)	Evidence	Notes
Clarity	2	screenshot-04.png	Amount shown in raw token units, not a human-readable quantity
Confirmation and consent	3	-	Unlimited approval requires a separate, distinctly styled modal

Dimension	Score (0-3)	Evidence	Notes
Revocability and recovery	1	ticket UX-118	No in-app revoke; user must leave the product to use an external allowance manager
Consistency and predictability	3	-	-
Warnings and error handling	2	screenshot-07.png	Generic “transaction failed” message gives no cause or next step

16.2 Internal Review vs. Third-Party and Community Audit

No single review type covers every failure mode at every cadence a product needs. Combine them deliberately rather than treating any one as sufficient on its own.

Review type	Best for	Typical cadence	Example
Internal design review	Catching issues before code exists	Every feature	Section 15.2 gate
Internal matrix audit	Full-scope scoring before a release ships	Every release that touches a signing or approval flow	Section 16.1 procedure
Third-party specialist audit	Independent validation, novel attack patterns, requirements a partner or regulator sets	Major releases, new wallet or connector integrations, at least annually	Contracted UX-security review firm
Community and bug bounty	Continuous, adversarial coverage at a volume no internal team can match	Ongoing	A bounty program scoped to include front-end and signing-flow issues, not contract code alone

Bug bounty platforms built for Web3, such as [Immunefi](#), [Code4rena](#), and [Cantina](#), primarily scope contract code, but most support extending scope to web assets and signing flows explicitly. A program that pays out for a contract reentrancy bug but not for a front-end that silently swaps a recipient address leaves the exact bug class this handbook exists to prevent outside its own incentive structure. When you write or renew a bounty scope, name the UX-security failure modes from Part I, Chapter 2 as in-scope, rather than leaving researchers to guess whether interface bugs qualify.

16.3 Scoring, Reporting, and Remediation Tracking

A finding that never leaves the audit report does not get fixed. Route every below-threshold dimension into the same issue tracker engineering already uses, with a severity derived mechanically from the score rather than from a case-by-case judgment call that different auditors will make differently.

Matrix finding	Example	Severity	Release impact
Dimension scores 0 on a fund-moving action	Unlimited approval granted with no confirmation step	Critical	Blocks release
Dimension scores 1 on a fund-moving action	Amount is shown but the asset is ambiguous or truncated	High	Blocks release unless waived in writing by the security owner
Dimension scores 0-1 on a non-fund action	Inconsistent terminology between settings and onboarding	Medium	Tracked, does not block
Cosmetic or informational deviation	Icon style mismatch between two flows	Low	Backlog

A minimal issue template keeps every finding comparable across auditors and audit cycles:

```

### UX Security Finding: <short title>
- **Dimension**: Confirmation and Consent
- **Score**: 1 / 3
- **Severity**: High
- **Flow**: Token approval (ERC-20 `approve`)
- **Evidence**: <screenshot or decoded-calldata reference>
- **Expected**: Explicit modal naming spender, amount, and asset before signing
- **Owner**:
- **Target release**:
- **Status**: Open

```

Track remediation the same way you would track a security vulnerability, because from the user's perspective it is one: a below-threshold score on a fund-moving action is a control gap, not a design opinion, and it should carry the same urgency in the tracker as a Critical finding from a contract audit.

16.4 Re-Audit Cadence and Regression Testing

A matrix score is a snapshot, and it decays. A flow that scored a perfect 3 across every dimension six months ago can silently regress when a dependency updates, a new wallet connector is added, or a redesign touches shared components. Define explicit triggers for re-audit rather than relying on someone remembering to schedule one.

Trigger	Re-audit scope
New wallet or connector integration	Full Confirmation and Consent and Consistency dimensions across the affected flows
New signature type or standard adopted (EIP-712 domain change, EIP-7702 delegation)	Full Clarity and Confirmation dimensions on flows using the new standard
Any incident or near-miss, regardless of severity	Full matrix on the affected flow, targeted spot-check on adjacent flows
Quarterly	Aggregate score refresh on the top flows by transaction volume

Trigger	Re-audit scope
Major visual redesign	Full Consistency and Predictability dimension across the product

Automate what regression testing can catch mechanically, so a re-audit trigger surfaces a known regression before a human auditor has to find it manually.

```
test('approve flow renders decoded spender, amount, and asset before signing', async
  ({ page }) => {
    await page.goto('/token/approve?spender=0xSpender&amount=max');
    const summary = await page.textContent('[data-testid="approval-summary"]');
    expect(summary).toContain('0xSpender'); // or a resolved, verified display name
    expect(summary).toMatch(/Unlimited|No limit/i); // unlimited must be labeled, never
    hidden
    await expect(page.locator('[data-testid="confirm-high-risk"]')).toBeVisible();
  });
```

A regression suite built from prior audit findings turns every past finding into a permanent guardrail: the bug that scored a 1 last quarter cannot silently return in next quarter's release without the test catching it first.

17. Iteration and User Feedback

The matrix is only as good as the signal that revises it. This chapter closes the loop between how real users behave, what incidents reveal, and how the matrix itself changes over time.

17.1 Usability Testing With a Security Lens

Standard usability testing asks whether a user can complete a task. A security lens on the same session asks a sharper question: does the user correctly understand what they are about to authorize before they authorize it? [Nielsen Norman Group's guidance on usability testing](#) holds that five participants surface most usability problems in a single round, and that principle applies directly to signing flows: run small, frequent rounds rather than one large study before launch. [ISO 9241-11](#) defines usability as effectiveness, efficiency, and satisfaction in achieving specified goals; for a security-critical flow, add a fourth criterion the standard does not name explicitly: whether the user's understanding of the action matches the action the system will actually perform.

A task-based script surfaces comprehension gaps that a simple “were you able to complete this?” question misses:

1. “Connect your wallet to this site. Before you click Approve, tell us out loud what you think will happen next.”

2. “You’ve been asked to approve a token spend. Without scrolling back up, tell us the amount and which contract will be able to spend it.”
3. “You approved a spend last week and no longer want that contract to have access. Try to undo that now.”
4. “This warning just appeared on screen. Read it out loud, then tell us what you would do next.”
5. “Something about this transaction looks different from ones you’ve signed before. What, if anything, would you do?”

Score the session against the matrix dimensions directly: a participant who cannot answer question 2 accurately is reporting a live Clarity failure, evidence-grade and reproducible, not a hypothetical one.

17.2 Incident and Near-Miss Feedback Loops

An incident report that only assigns blame teaches a team to hide near-misses rather than report them. A blameless postmortem, factual and focused on the flow rather than the individual, produces evidence the matrix can act on.

Field	Purpose
What happened	Factual timeline, free of blame language
What the user saw	Exact UI state at each step, with screenshots where available
What the matrix would have scored	Retroactive score against each dimension
Why it wasn’t caught	Which gate, test, or review should have caught it and did not
Matrix or process change	The concrete change proposed as a result
Owner and date	Who is accountable for shipping the fix, and by when

The Bybit exchange’s February 2025 loss, covered as a threat pattern in Part I, Section 2.4, is the reference case for why this loop matters: the failure was not a missing warning on a single screen but a signing flow where what several multisig signers saw did not match what they were actually authorizing, and no feedback mechanism in that flow surfaced the drift before it was exploited at scale. A near-miss log that captures smaller versions of the same drift, a support ticket describing a transaction that “looked wrong but went through fine,” a signer who paused but signed anyway, is the earliest and cheapest place to catch that pattern, well before it reaches the severity of an incident.

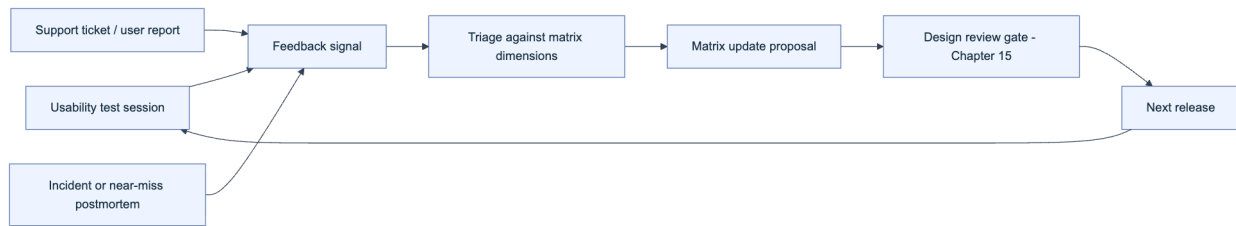


Figure 4.2. Three independent feedback sources feed one triage step, so a pattern visible only across usability testing, support, and incident data is not missed because it lived below any single source's reporting threshold.

17.3 Evolving the Matrix as Threats Change

The matrix defined in Part II is not fixed. New signing standards, new delegation mechanisms, and new phishing techniques appear faster than any static checklist can anticipate, and a matrix that never changes eventually scores a genuinely dangerous flow as passing simply because nobody wrote a criterion for the new attack. MITRE's [AADAPT framework](#) catalogs adversary techniques against digital asset platforms in the same spirit as ATT&CK for enterprise systems, and it is a useful external source of new criteria: when AADAPT documents a technique your matrix has no dimension or sub-criterion for, that is a concrete signal to open a revision, not a hypothetical one.

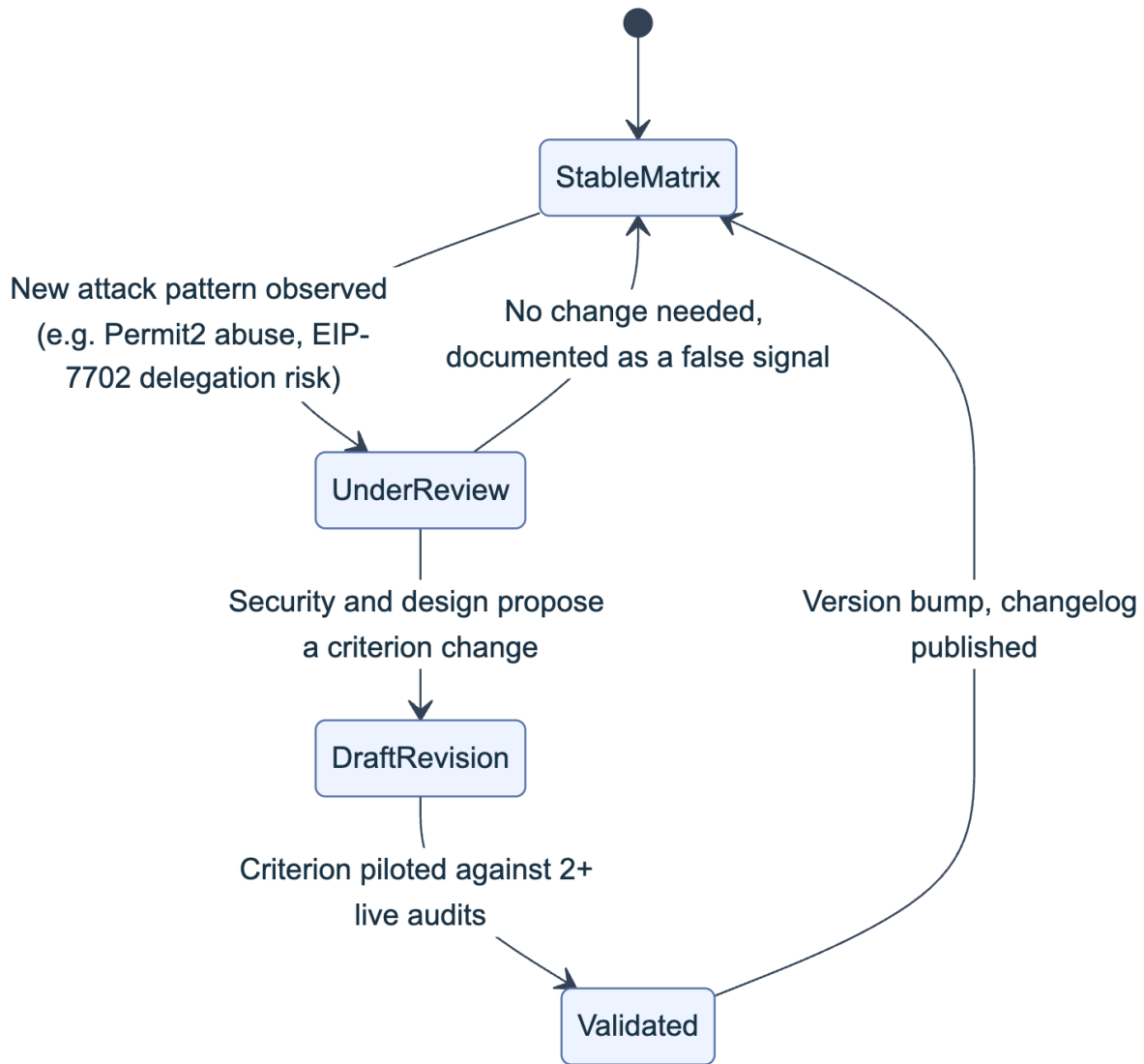


Figure 4.3. The matrix's own revision lifecycle. A proposed change is piloted against real audits before it becomes binding, so the matrix does not accumulate untested criteria.

Version the matrix like any other piece of shared infrastructure, with a changelog that states what changed and why:

Matrix changelog

v1.3 (this quarter)

- Added: Confirmation and Consent criterion for EIP-7702 delegation grants. A delegate that gains code-execution rights over an externally owned account (EOA) must be shown as a "smart account upgrade," not rendered as a routine signature request.
- Tightened: Revocability scoring now requires an in-app revoke path; a link to an external allowance manager alone no longer scores above 1.

A trigger table keeps revisions deliberate rather than reactive to every single support ticket:

Trigger	Matrix action
New standard adopted industry-wide (EIP-7702, Permit2, a new signature scheme)	New criterion drafted under the relevant dimension, or under Chapter 9's expandable dimensions
Novel attack pattern observed, internally or via a public incident	New criterion or dimension drafted, piloted, then merged per Figure 4.3
Two consecutive quarters of near-perfect scores across all flows on one dimension	Criteria tightened so the ceiling stays meaningful
Engineering reports a recurring false positive against a criterion	Criterion clarified or narrowed, with the reasoning recorded in the changelog

Key controls for Part 4

- Gate every design at a defined review checkpoint before implementation starts, with a logged pass, fail, or written exemption.
- Assign accountability for security-critical matrix dimensions to security, not to whichever role is under the most deadline pressure.
- Automate the cheap, high-confidence checks (unguarded unlimited approvals, calldata-to-summary mismatches) in CI so they block a merge instead of waiting for a human audit.
- Run matrix audits on a fixed procedure with recorded evidence, so scores are reproducible across auditors and comparable across release cycles.
- Combine internal review, third-party audit, and community bug bounty scoped to include front-end and signing flows, not contract code alone.
- Treat a below-threshold score on a fund-moving action with the same urgency as a Critical contract finding, and track it to closure in the same system.
- Re-audit on defined triggers (new connector, new signature standard, incident, quarterly cadence) rather than only before a major launch.
- Feed usability testing, support signals, and incident postmortems into one triage step, and version the matrix itself with a changelog when a genuine gap is found.

Part V: Appendices

This part collects the reference material the rest of the handbook assumes a reader keeps within reach. Appendix A fixes the vocabulary so that a term like *blind signing* or *revocation* means the same thing in Chapter 2 as it does in Chapter 12. Appendix B is the condensed, printable form of the UX (user experience) Security Scoring Matrix built out across Part II, meant for a design review where nobody has time to reread six chapters. Appendix C turns the checklists scattered through Parts III and IV into copy-ready artifacts, and Appendix D points to the full source list rather than repeating every citation already given at the point of its claim.

18. Appendix A: Glossary

Entries below are grouped by the domain area where they carry the most weight, not strict alphabetical order, because a reader reviewing a wallet's signing flow and a reader auditing an allowance screen reach for different clusters of vocabulary. Within each group, entries run alphabetically, with the acronym expanded on first appearance and the relevant chapter named so you can jump to the fuller treatment. Where an external standard, Ethereum Improvement Proposal (EIP), or documented incident defines the term, the entry cites that source directly.

18.1 Threat and Deception Terms

These terms describe how an attacker manipulates what a user sees and signs, the core threat model built in Chapter 2.

- **Approval phishing:** tricking a user into signing a token approval or `permit` message that grants an attacker-controlled address spending rights over the user's tokens, rather than stealing a private key directly. Named as its own category, UI/UX Spoofing and Approval Phishing (WA06), in the [OWASP Web3 Attack Vectors Top 15](#). See Chapter 2.2.
- **Blind signing:** approving a transaction or message without the wallet decoding and displaying its actual effect, forcing the user to trust the requesting dApp's (decentralized application's) representation of what they are signing rather than verified calldata. Central to the pattern behind the February 2025 Bybit theft. See Chapter 2.4 and Chapter 10.2.

- **Calldata/UI mismatch:** a discrepancy between the transaction data a wallet will actually broadcast and the recipient, amount, or function the requesting interface displays, whether from a bug or deliberate spoofing. See Chapter 2.7.
- **Drainer / Drainer-as-a-Service (DaaS):** malicious code, often sold as a hosted kit to affiliates, that automates draining a connected wallet once a victim signs a malicious approval. [Scam Sniffer's 2025 phishing research](#) tracks named kits such as Inferno Drainer and reports 2025 phishing losses near \$84 million, down 83 percent year over year as wallet-level warnings and blocklists matured. See Chapter 2.2.
- **Ice phishing:** a term for approval-phishing attacks that never touch a private key at all, coined to distinguish them from classic seed-phrase theft; the victim signs away spending rights while believing they are approving a routine transaction.
- **Malicious dApp impersonation:** a fraudulent front end that copies a legitimate protocol's branding and domain-adjacent naming to collect approvals or signatures, the failure mode the WalletConnect Verify API (Chapter 11.1) is built to catch.
- **Multisig hijacking:** compromising the signing or review interface a multisig operator relies on, so that operators approve a transaction whose real effect differs from what they reviewed, without any signer's key being stolen. Listed as WA01 in the [Web3 Attack Vectors Top 15](#) and the mechanism behind the Bybit theft. See Chapter 2.4.
- **UI/UX spoofing:** the umbrella term for any interface manipulation, fake confirmation dialog, mislabeled button, or altered display value, that causes a user to consent to something other than what they believe they are consenting to. See Chapter 2.5.
- **Wallet drain:** the end state of a successful phishing, drainer, or blind-signing attack, in which an attacker-controlled address receives a user's tokens or gains an approval sufficient to take them on a later transaction.

18.2 Approval, Signature, and Standard Terms

These terms describe the mechanisms by which a user grants a contract or address rights over their tokens, and the standards that govern how that consent is captured, covered across Chapter 2.3 and Chapter 12.

- **Allowance:** the amount of a token, tracked in the token contract's storage, that an owner has authorized a spender address to transfer on their behalf, set through the legacy [ERC-20 \(Ethereum Request for Comment\)](#) `approve` function.
- **EIP-712:** the Ethereum standard for hashing and signing typed structured data, letting a wallet display a message's fields (recipient, amount, deadline) instead of an opaque hex string. First proposed in 2017 and central to human-readable signature review. See [EIP-712](#) and Chapter 10.3.
- **EIP-2612 (`permit`):** an extension to the ERC-20 standard adding a `permit` function that lets a token owner set an allowance through an off-chain EIP-712 signature instead of an

on-chain `approve` transaction, removing the need to hold gas token to grant an approval. See [EIP-2612](#) and Chapter 2.3.

- **EIP-7702:** a standard letting an externally owned account (EOA) temporarily or permanently delegate its code execution to a smart contract, enabling batched transactions, sponsored gas, and sub-key permission schemes while the account keeps its original address. Reached Final status and activated on Ethereum mainnet as part of the Pectra upgrade on [May 7, 2025](#). See [EIP-7702](#) and Chapter 10.4.
- **Nonce:** a strictly increasing counter tied to a signer or contract, included in a signed message to prevent the same signature from being replayed a second time. Relevant to both transaction ordering and `permit` signature validity.
- **Permit2:** a unified, immutable contract published by Uniswap Labs that layers signature-based (`SignatureTransfer`) and allowance-based (`AllowanceTransfer`) authorization over any ERC-20 token, adding expiry timestamps and per-spender amount caps that neither legacy `approve` nor EIP-2612 `permit` enforce on their own. See the [Permit2 contract and documentation](#) and Chapter 2.3.
- **Revocation:** setting an existing allowance or Permit2 authorization back to zero (or letting it expire, for Permit2's time-bound grants), removing a spender's ability to move the owner's tokens. See Chapter 6.1 and Chapter 12.1.
- **Unlimited allowance:** an approval set to the maximum representable value (`type(uint256).max`) rather than the amount actually needed for a transaction, a widespread convenience pattern that leaves a user permanently exposed to any future compromise of the approved spender. See Chapter 2.2.

18.3 Wallet and Account Terms

These terms describe the account models and confirmation surfaces a user interacts with directly, covered in Chapter 10.

- **Account abstraction:** replacing an EOA's fixed signature-verification logic with programmable smart contract logic, enabling features such as social recovery, session keys, and gas sponsorship. Standardized for existing EVM (Ethereum Virtual Machine) chains without protocol changes by [ERC-4337](#) and extended at the protocol level by EIP-7702. See Chapter 10.5.
- **Clear signing:** a hardware wallet's ability to parse a transaction or typed-data message and render its actual effect (recipient, amount, contract call) on the device's own trusted screen, as opposed to blind signing a value the host computer supplied. See Chapter 10.2.
- **Cold wallet:** a wallet whose private key material never touches an internet-connected device, typically a hardware wallet or an air-gapped signer. See Chapter 10.1.
- **Custodial wallet:** a wallet where a third party (an exchange or service provider) holds the private keys on the user's behalf, trading self-custody for a recovery path if the user loses access. See Chapter 10.1.

- **EOA (externally owned account):** an Ethereum account controlled directly by a private key, as opposed to a smart contract account whose logic governs its behavior.
- **Hot wallet:** a wallet whose keys are held on an internet-connected device, trading resistance-to-remote-compromise for signing convenience. See Chapter 10.1.
- **MPC (multi-party computation) wallet:** a wallet that splits a private key into shares held by separate parties or devices, such that no single share alone can produce a valid signature, used by some custodial and institutional providers as an alternative to a single hardware key.
- **Session key:** a limited-scope, often time-bound signing key that an account abstraction wallet issues to an application so it can execute a narrow set of pre-approved actions without prompting the user for every transaction. See Chapter 10.5.

18.4 Scoring, Standards, and Governance Terms

These terms describe the frameworks that structure the threat model in Chapter 2 and the matrix in Part II.

- **AADAPT (Adversarial Actions in Digital Asset Payment Technologies):** a MITRE threat framework modeled on ATT&CK, cataloging adversary tactics specific to digital-asset and payment systems. See the [public matrix repository](#).
- **Aggregate score:** the weighted roll-up of a product's scores across every UX Security Scoring Matrix dimension, banded into a risk category for reporting and prioritization. See Chapter 9.3 and Appendix B.19.3.
- **Dimension:** one of the matrix's six evaluation axes, clarity, confirmation and consent, revocability and recovery, consistency and predictability, warnings and error handling, and accessibility and inclusivity, each scored independently before being aggregated. See Chapters 4 through 9.
- **MITRE ATT&CK:** a globally accessible knowledge base of adversary tactics and techniques drawn from real-world observations, used across the security industry to structure threat models. See [attack.mitre.org](#).
- **OWASP SC (Smart Contract) Top 10:** OWASP's ranked list of the most critical smart contract vulnerability categories; none of its ten categories address interface-layer risk directly, which is the gap this handbook and the companion Web3 Attack Vectors Top 15 fill. See [scs.owasp.org/sctop10](#) and Chapter 1.3.
- **SCSVS (Smart Contract Security Verification Standard):** OWASP's standard defining verifiable security requirements for smart contract systems; this handbook's scoring matrix is its interface-layer counterpart. See [scs.owasp.org](#) and Chapter 1.3.
- **SCSTG (Smart Contract Security Testing Guide):** OWASP's companion guide describing test procedures for smart contract security, extended by Part IV's matrix-based review process. See [scs.owasp.org](#) and Chapter 1.3.

- **SCWE (Smart Contract Weakness Enumeration):** OWASP's catalog of smart contract weakness classes, which stops at the contract boundary; the phishing, allowance-abuse, and blind-signing failure modes in Chapter 2 fill the human-computer-interaction space it leaves open. See scs.owasp.org and Chapter 1.3.
- **STRIDE:** a threat-modeling mnemonic, spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege, introduced by Loren Kohnfelder and Praerit Garg at Microsoft in 1999 and applicable to interface-layer threats such as spoofed confirmation dialogs (spoofing) and altered display values (tampering).
- **Web3 Attack Vectors Top 15:** an OWASP catalog ranking the most significant non-contract Web3 risks, including Multisig Hijacking (WA01), Drainer Malware and Drainer-as-a-Service (WA04), and UI/UX Spoofing and Approval Phishing (WA06), the three categories most directly addressed by this handbook. See scs.owasp.org/sctop10/Web3-Attack-Vectors-Top15 and Chapter 1.3.

19. Appendix B: UX Security Scoring Matrix (Reference)

Part II builds the UX Security Scoring Matrix dimension by dimension, with worked criteria and examples for each. This appendix is the condensed, single-page version: pull it out for a design review or a pre-launch audit where the reviewer needs the scoring structure in front of them without re-reading six chapters. Use it alongside, not instead of, the full dimension chapters when a score needs justification beyond a number.

19.1 Matrix at a Glance

Each dimension is scored independently on the same 0-to-3 scale defined in Chapter 3.3, then combined per Section 19.3. The table below names the dimension, the chapter with its full criteria, and a one-line anchor for what a 0 and a 3 look like in practice.

Dimension	Chapter	Score 0 looks like	Score 3 looks like
Transaction and Action Clarity	4	Raw hex or contract address shown with no plain-language summary	Recipient, amount, and asset shown in plain language, cross-checked against calldata
Confirmation and Consent	5	High-risk actions (unlimited approval, ownership transfer) execute with a single generic click	Distinct, explicit confirmation step for each high-risk action, with the specific risk named
Revocability and Recovery	6	No in-product way to view or revoke a prior approval	One-click revocation surfaced in the product itself, with a documented recovery path for lost access
Consistency and Predictability	7	Terminology and navigation differ across flows or between web and mobile	Identical terms, layouts, and outcomes across every entry point

Dimension	Chapter	Score 0 looks like	Score 3 looks like
Warnings and Error Handling	8	Errors surface as raw revert strings or fail silently	Warnings are risk-specific, actionable, and appear before signing, not after
Accessibility and Inclusivity	9.1	Signing flow unusable with a screen reader or on a small viewport	Meets WCAG (Web Content Accessibility Guidelines) AA for the signing and approval path

19.2 Quick-Score Worksheet

Copy this table into a review ticket or design document. Score each dimension from the criteria in its chapter, cite the specific evidence (a screenshot, a flow recording, a line of copy), and note the reviewer and date so the score is traceable when the product changes.

UX Security Scoring Matrix – Quick Score

Dimension	Score (0–3)	Evidence	Reviewer	Date
Transaction and Action Clarity (Ch. 4)				
Confirmation and Consent (Ch. 5)				
Revocability and Recovery (Ch. 6)				
Consistency and Predictability (Ch. 7)				
Warnings and Error Handling (Ch. 8)				
Accessibility and Inclusivity (Ch. 9.1)				

Weighted aggregate: ____ / 18 (see 19.3 for banding)

Feature or product reviewed: _____

Next review due: _____

Weight the dimensions equally by default (sum the six raw scores out of a possible 18), as Chapter 3.2 recommends for a first pass. A team that has already suffered an incident in one dimension, an allowance-abuse loss, for instance, may apply a higher weight to Revocability and Recovery for subsequent reviews; document any deviation from equal weighting next to the aggregate so a later reviewer understands why the number differs from the default formula.

19.3 Interpreting the Aggregate Score

A raw number is only useful once it is banded into an action. The table and diagram below apply the aggregation and reporting method from Chapter 9.3 to the six-dimension, 18-point scale above.

Aggregate score	Band	Recommended action
0-6	Critical risk	Block launch; escalate to security review before further design work
7-10	High risk	Fix the lowest-scoring dimension before launch; schedule re-score within one sprint
11-14	Needs improvement	Ship with a documented remediation plan; re-score at the next release

Aggregate score	Band	Recommended action
15-16	Solid	Note residual gaps; re-score on the standard cadence (Chapter 16)
17-18	Exemplary	Candidate for a documented pattern other teams can reuse

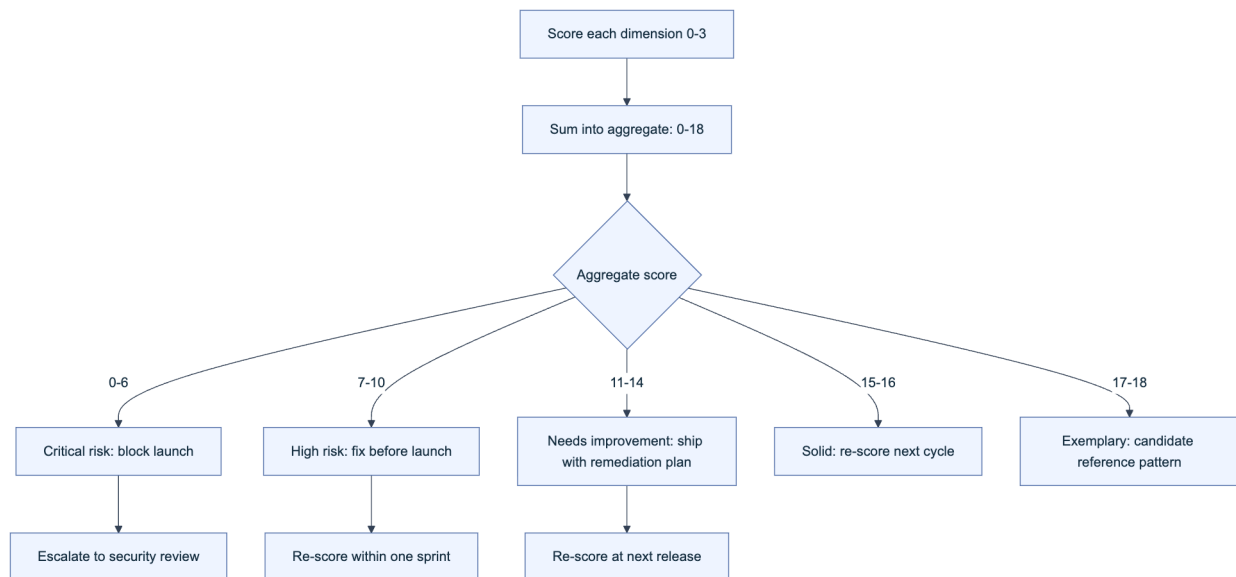


Figure 1. Scoring-to-action flow. The band, not the raw number; is what a release gate should key off; since a single catastrophic dimension (for example, no revocation path) can sit inside an otherwise “Solid” aggregate and still deserves the higher band’s response; treat the banding table as a floor, not a substitute for reading the lowest individual score.

20. Appendix C: Checklist and Quick Wins

Part III turns the matrix into product-level practice across wallet, connection, allowance, notification, and onboarding flows; Part IV covers integrating that practice into a development lifecycle. This appendix collects the copy-ready outputs of both: a pre-launch checklist, a quick-wins table for teams that cannot fix everything at once, and a decision flow for picking the right artifact for the moment you are in.

20.1 Pre-Launch UX Security Checklist

Run this checklist before any release that touches a signing, connection, approval, or notification flow, and again on the cadence set in Chapter 16. It mirrors the five best-practice chapters in Part III so nothing is checked from memory.

Pre-Launch UX Security Checklist

Wallet and Signing (Ch. 10)

- ☐ EIP-712 typed data used for every off-chain signature request; no raw hex presented for a signature a user is expected to understand
- ☐ Hardware wallet flows tested for clear signing on the actual device screen, not only in the host application's UI
- ☐ Any EIP-7702 delegation or account abstraction flow discloses what permissions the delegated contract receives, in plain language, before the user signs

dApp and Connection (Ch. 11)

- ☐ WalletConnect Verify API (or equivalent domain-verification signal) is enabled and its warning state is visibly surfaced, not silently logged
- ☐ Primary domain is bookmarked and communicated through a verified, out-of-band channel (Ch. 11.2)

Approval and Allowance (Ch. 12)

- ☐ Default approval amount is scoped to the transaction, not unlimited
- ☐ Every approval has a visible expiry or is revocable in one step from within the product
- ☐ Permit2 or EIP-2612 signature requests show spender, amount, and expiry before the user signs

Notification and Alert (Ch. 13)

- ☐ High-risk actions (new spender approval, ownership change, large transfer) trigger a distinct, non-dismissible-by-accident warning
- ☐ Notification copy is reviewed against the Warnings and Error Handling dimension (Ch. 8) for specificity and actionability

Onboarding and Education (Ch. 14)

- ☐ First-time users see a concrete explanation of what a signature and an approval authorize, in the flow, not only in a linked help article
- ☐ Recovery and support paths are documented and reachable from the signing screen itself

20.2 Quick Wins by Effort and Impact

Not every gap can close before a release ships. Use this table to prioritize when the full checklist surfaces more findings than the current cycle can absorb; sort by lowest effort within the highest-impact rows first.

Quick win	Effort	Impact	Dimension improved	Reference
Cap default approval amount to the transaction value instead of unlimited	Low	High	Confirmation and Consent	Ch. 12.1
Add a one-click revoke link next to every active allowance in-product	Low	High	Revocability and Recovery	Ch. 6.1
Replace raw revert strings with a mapped, plain-language error message	Low	Medium	Warnings and Error Handling	Ch. 8.2
Enable WalletConnect Verify API and surface its warning state in the connect modal	Low	High	Clarity / Confirmation	Ch. 11.1
Add an explicit confirmation step (separate from the wallet's own prompt) for ownership transfer or unlimited approval	Medium	High	Confirmation and Consent	Ch. 5.1
Standardize terminology for "approve," "allow," and "authorize" across web and mobile	Medium	Medium	Consistency and Predictability	Ch. 7.1
Add plain-language summaries above EIP-712 typed data fields	Medium	High	Transaction and Action Clarity	Ch. 4.2

Quick win	Effort	Impact	Dimension improved	Reference
Audit signing and approval flows against WCAG AA	High	Medium	Accessibility and Inclusivity	Ch. 9.1

20.3 Template and Checklist Selection Flow

The right artifact depends on where you are in the release or incident cycle. Use the flow below to pick a starting point; a single release touching signing, connection, and allowance UX will often use more than one branch.

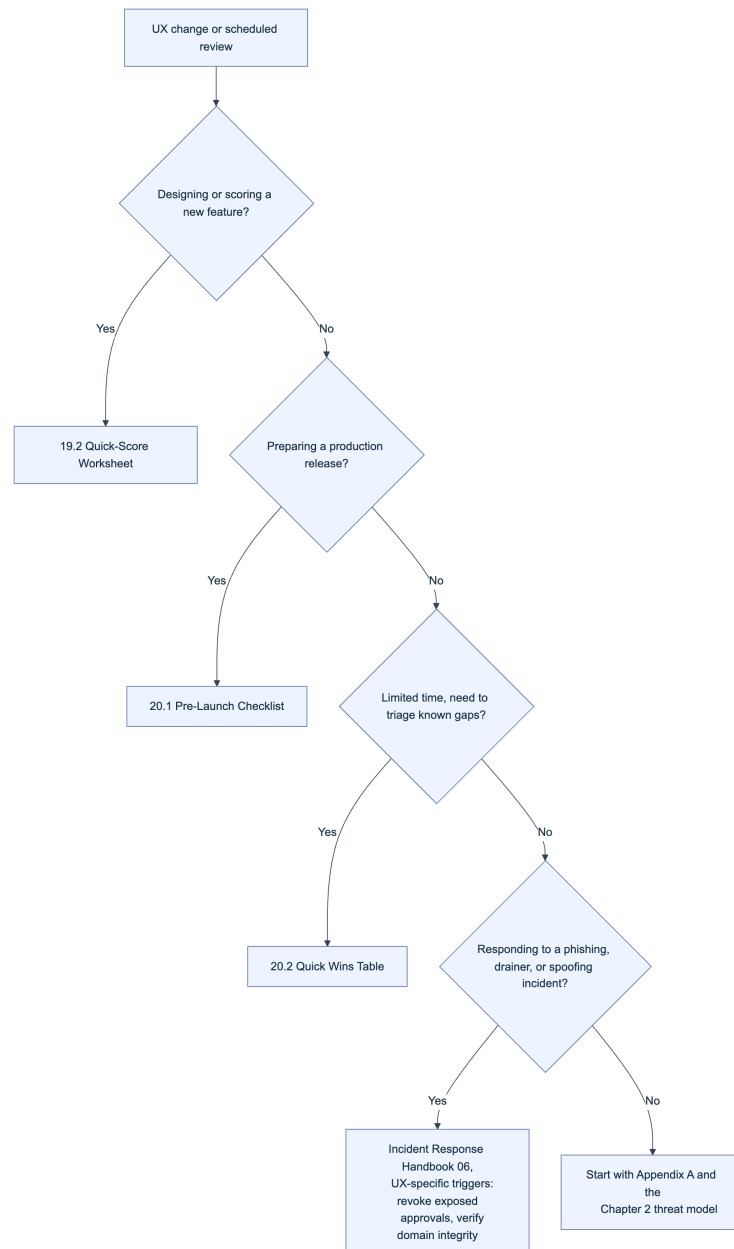


Figure 2. Artifact selection by trigger scenario. A release that changes both the signing flow and the allowance screen runs both 20.1 and, if time is short, prioritizes with 20.2.

21. Appendix D: References and Further Reading

Every claim in this handbook that names a standard, a version, a date, or an incident is cited inline at the point it is made, following the convention set in Part I. Rather than duplicate that source list here, this appendix points to `references.md`, the deduplicated bibliography for the full handbook, grouped into Standards and Frameworks (the OWASP SC Top 10, the Web3 Attack Vectors Top 15, SCSVS, SCSTG, SCWE, EIP-712, EIP-2612, EIP-7702, ERC-4337, and MITRE AADAPT), Incidents and Case Studies (the February 2025 Bybit theft and the [FBI Internet Crime Complaint Center advisory on the TraderTraitor activity behind it](#), alongside documented approval-phishing and drainer campaigns tracked by Scam Sniffer), Tools (the WalletConnect Verify API, Permit2, hardware wallet clear-signing implementations, and allowance-revocation dashboards), and Further Reading (research write-ups and companion material that goes deeper than this handbook's scope allows).

If a link in the body of any Part goes stale, check `references.md` for an updated URL or an archive copy before assuming the underlying claim no longer holds. When you extend this handbook, add the new source to `references.md` in the same change rather than letting citations accumulate only in the body text, so the bibliography stays a complete index rather than a partial one.

Key controls for Part 5

- Treat Appendix A as the canonical vocabulary; link to it from design tickets and incident write-ups instead of redefining a term inline each time.
- Keep the Quick-Score Worksheet (19.2) attached to every feature design doc that touches signing, connection, or approval flows.
- Run the Pre-Launch UX Security Checklist (20.1) before every release, not only after a phishing or drainer incident prompts one.
- Use the Quick Wins table (20.2) to sequence remediation when the full checklist surfaces more gaps than the current cycle can close, prioritizing low-effort, high-impact rows first.
- Score revocability and confirmation for unlimited or long-lived approvals with the same weight as clarity; a single missing revoke path can sit inside an otherwise “Solid” aggregate (19.3) and still represent the finding that matters most.
- Add every new citation to `references.md` in the same change that introduces it (Appendix D).

Appendix C: References and Further Reading

This appendix collects every external source cited across the UX Security Handbook, grouped by category for easy lookup.

Standards and Frameworks

- MITRE AADAPT Framework
- MITRE ATT&CK
- MITRE AADAPT public matrix repository, GitHub
- NIST SP 800-218, Secure Software Development Framework (SSDF)
- EIP-20: Token Standard
- EIP-712: Typed Structured Data Hashing and Signing
- EIP-2612: Permit Extension for EIP-20
- ERC-4337: Account Abstraction
- EIP-7702: Set EOA Account Code
- ERC-7730: Clear Signing Format
- OWASP Software Assurance Maturity Model (SAMM)
- OWASP Smart Contract Security (SCS), including SCSTG, SCSVS, and SCWE
- OWASP Smart Contract Top 10: 2026
- OWASP Web3 Attack Vectors Top 15
- ISO 9241-11: Ergonomics of Human-System Interaction
- W3C Web Content Accessibility Guidelines (WCAG) 2.2

Incidents and Case Studies

- Bybit background, Wikipedia
- Curve Finance loses \$570k due to DNS compromise, CryptoTimes
- FBI Internet Crime Complaint Center (IC3), PSA250226 - TraderTraitor advisory
- Scam Sniffer 2025 phishing report
- Sygnia investigation into the Bybit hack

Tools and Documentation

- Blockaid
- Cantina

- [Code4rena](#)
- [Immunefi](#)
- [Permit2 overview, Uniswap developer documentation](#)
- [Permit2 overview, Uniswap contracts documentation](#)
- [Permit2 contract and documentation, GitHub](#)
- [Reown documentation](#)
- [WalletConnect Verify API documentation](#)
- [Revoke.cash](#)
- [Ledger, Clear Signing](#)

Further Reading

- [Ethereum Foundation, Pectra roadmap](#)
- [Deceptive Design taxonomy](#)
- [Nielsen Norman Group, “10 Usability Heuristics for User Interface Design”](#)
- [Nielsen Norman Group, “How to Rate the Severity of Usability Problems”](#)
- [Nielsen Norman Group, “Usability Testing 101”](#)
- [Crying Wolf: An Empirical Study of SSL Warning Effectiveness, USENIX Security 2009](#)